



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

อัลกอริทึมย้อนรอย

(Backtracking Algorithm)

ปรับปรุงจาก เอกสารของ ผศ. ดร.ลือพล พิพานเมฆาภรณ์

ผศ. ดร.สรวิ รัตนสัญญา

san.r@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

- ปัญหาการเรียงสับเปลี่ยน (Permutation) และปัญหาการจัด (Combinatorics)
 - ตัวอย่างปัญหาการเรียงสับเปลี่ยน
 - ตัวอย่างปัญหาการจัด
- การค้นปริภูมิสถานะ (State space searching)
- การตะลุยหาคำตอบ (Brute-force)
- อัลกอริทึมย้อนรอย (Backtracking algorithm)

ทบทวนการเรียงสับเปลี่ยนและการจัดกลุ่ม

	With repetition	Without repetition
❖ Permutation: Order is important.	n^r	$n_{Pr} = \frac{n!}{(n-r)!}$
MS Excel Formula >>	PERMUTATIONA(n,r)	PERMUT(n,r)
❖ Combination: Order is NOT important.	$\frac{(r+n-1)!}{r!(n-1)!}$	$n_{Cr} = \frac{n!}{(n-r)!r!}$
MS Excel Formula >>	COMBINA(n,r)	COMBIN(n,r)

QG

$$\binom{n+k-1}{k} = \text{Stars and Bars} = \binom{n+k-1}{n-1}$$



Stars and Bars



PERMUTATIONS / COMBINATIONS SUMMARY

ปัญหาการเรียงสลับเปลี่ยนและปัญหาเชิงการจัด (Permutation and Combinatorial problems)

- พบได้บ่อยในชีวิตประจำวัน
- ลักษณะของปัญหาเชิงการจัด
 - คำตอบของปัญหาอยู่ในรูปเซตของทางเลือก
 - เป็นเซตแบบมีลำดับ หรือไม่มีลำดับ
 - มีคำตอบที่เป็นไปได้มากกว่าหนึ่งคำตอบ ขึ้นกับเงื่อนไขของปัญหานั้น ๆ
 - เลือกคำตอบที่เหมาะสมที่สุด (Optimal solution)



ปัญหาการเรียงสลับเปลี่ยน (Permutation) และ ปัญหาการจัด (Combinatorics)

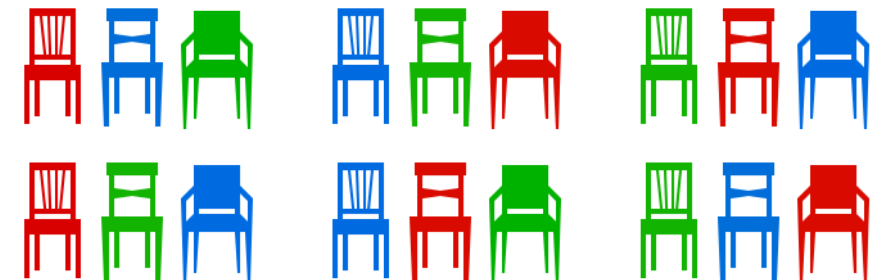
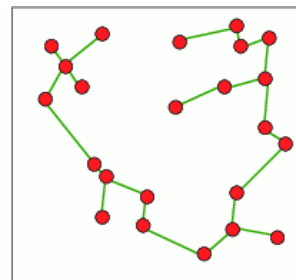
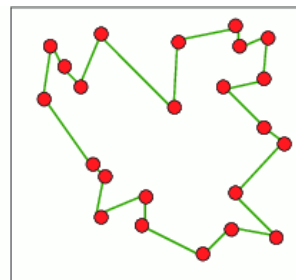
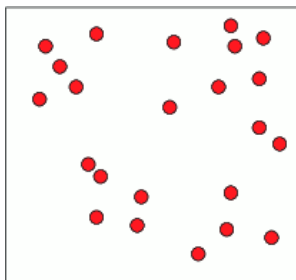
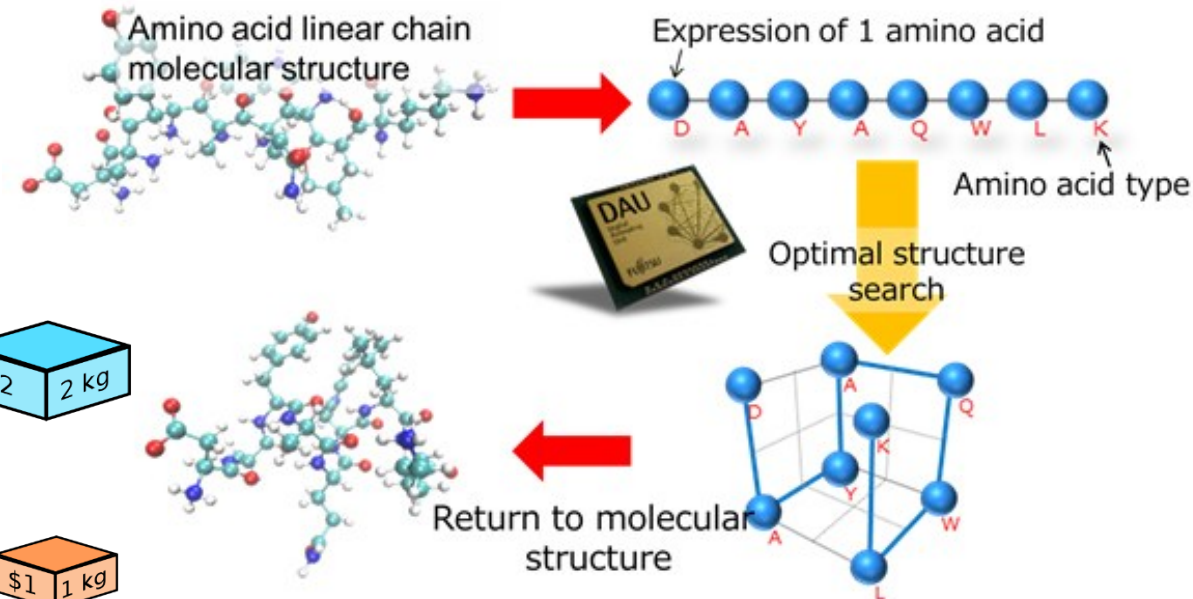
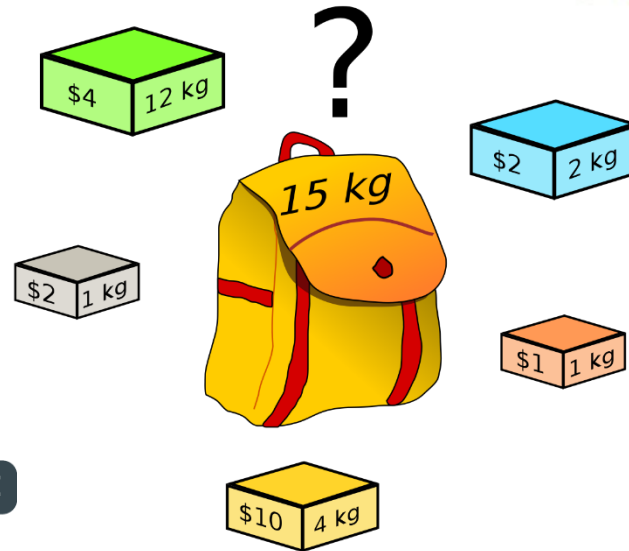
- ปัญหาการเรียงสลับเปลี่ยน (Permutation problem) เป็นปัญหาการจัดเรียง (order) สมาชิก
- ปัญหาการจัด (Combinatorial problem) เป็นปัญหาที่เกี่ยวข้องกับการนับ (counting) และการเลือก (selection) สมาชิก
- โดยทั่วไปปัญหาเหล่านี้มักมาพร้อมกับเงื่อนไขบังคับ (constraint) ของปัญหา
- ปัญหาเหล่านี้มักต้องการคำตอบที่ไม่ขัดแย้งกับเงื่อนไขบังคับ

ปัญหาการเรียงสลับเปลี่ยนและปัญหาเชิงการจัด (Permutation and Combinatorial problems)

■ ตัวอย่างของปัญหา



Distance from city A to E



ตัวอย่างปัญหาการเรียงสับเปลี่ยน

- มีเก้าอี้ 3 ตัววางเรียงติดกัน มีเด็ก 3 คน เป็นเด็กชาย 1 คนและเด็กหญิง 2 คน จงหาจำนวนวิธีทั้งหมดในการจัดเด็กทั้งหมดนั่งเก้าอี้ โดยไม่อนุญาตให้เด็กผู้หญิงสองคนนั่งติดกัน



Permutation

```

29 int main() {
30     int n = 3, i;
31     int x[n+1];
32
33     for(i=1; i<=n; i++)
34     { x[i] = i;
35     }
36     permute(x, 1, n);
37     return 0;
38 }
1  #include <stdio.h>
2  void print(int x[], int n)
3  {
4      for(int i=1; i<=n; i++)
5      { printf("%d ", x[i]);
6      }
7      printf("\n");
8  }

```

```

17 void permute(int x[], int l, int r)
18 { if ( l == r)
19     print(x, r);
20 else {
21     for(int i = l; i<=r ; i++)
22     { swap(&x[l], &x[i]);
23       permute(x, l+1, r);
24       swap(&x[l], &x[i]);
25     }
26 }
27 }

```

```

10 void swap(int *x, int *y)
11 { int tmp = *x;
12   *x = *y;
13   *y = tmp;
14 }

```

ตัวอย่างปัญหาการจัด ผลรวมของเซตย่อย (Subset Sum)

- กำหนดให้ $A = \{25, 10, 9, 2, 1\}$ จงหาเซตย่อยทั้งหมดของ A ที่มีผลรวมของสมาชิกเท่ากับ **12**
 - **$\{10, 2\}$** และ **$\{9, 2, 1\}$**
- หรืออาจจะถามว่าเซตย่อยที่เล็กที่สุดที่มีผลรวมเท่ากับ 12 คือเซตใด
 - กรณีนี้คำตอบจะมีเพียง **$\{10, 2\}$** เท่านั้น

Subset

```

24 int main()
25 {   int n = 3;
26     int x[n + 1];
27     Subset1(x, 0, n);
28     return 0;
29 }

```

```

1  #include <stdio.h>
2
3  int A[] = {6, 7, 8};
4

```

```

13 void Subset1(int x[], int l, int r)
14 {   if (l == r)
15     print_sol(x, r);
16     else
17     {   x[l+1] = 1;
18         Subset1(x, l+1, r);
19         x[l+1] = 0;
20         Subset1(x, l+1, r);
21     }
22 }

```

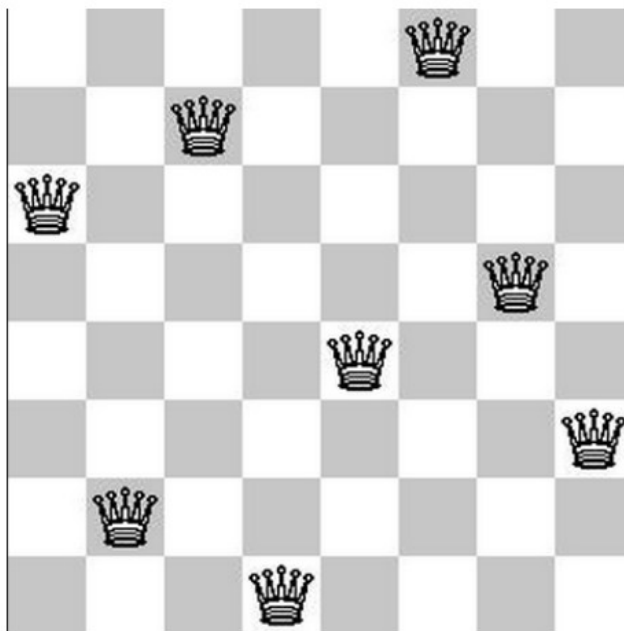
```

5 void print_sol(int x[], int n)
6 {   for (int i = 1; i <= n; i++)
7     {   if (x[i]== 1)
8         printf("%d ", A[i-1]);
9     }
10    printf("\n");
11 }

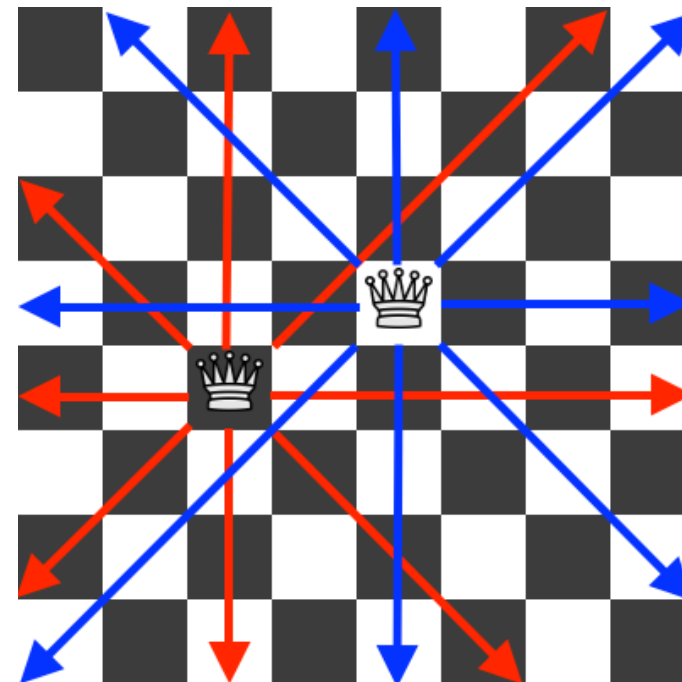
```

ปัญหา N ควีน

- จงหาจำนวนวิธีทั้งหมดที่สามารถวางควีน (queen) 8 ตัว ลงบนบอร์ดขนาด 8 แถว 8 หลัก โดยไม่มีควีนใดที่วางอยู่ในแนวเดียวกันในทุกทิศทาง



8 ควีน



Sudoku เกมลับอรรถ

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

กระดานเกมส์เริ่มต้น

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

กระดานคำตอบ

ปัญหาถุงเป้ (Knapsack problem)

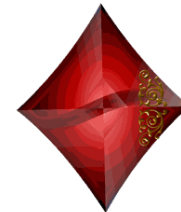
- ต้องการเลือกไอเทม (item) n ชิ้น ซึ่งมีน้ำหนัก (weight) และมูลค่า (profit) แตกต่างกันไป ใส่ลงในเป้ (knapsack) ซึ่งรับน้ำหนักได้ W กิโลกรัม โดยให้มูลค่ารวมของไอเทมในเป้มีค่ามากที่สุด และถุงไม่ขาด



Wt. = 5
Value = 10



Wt. = 3
Value = 20



Wt. = 8
Value = 25



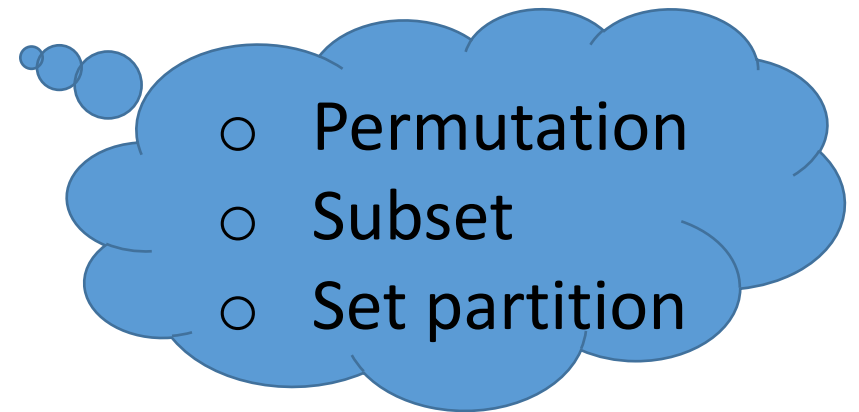
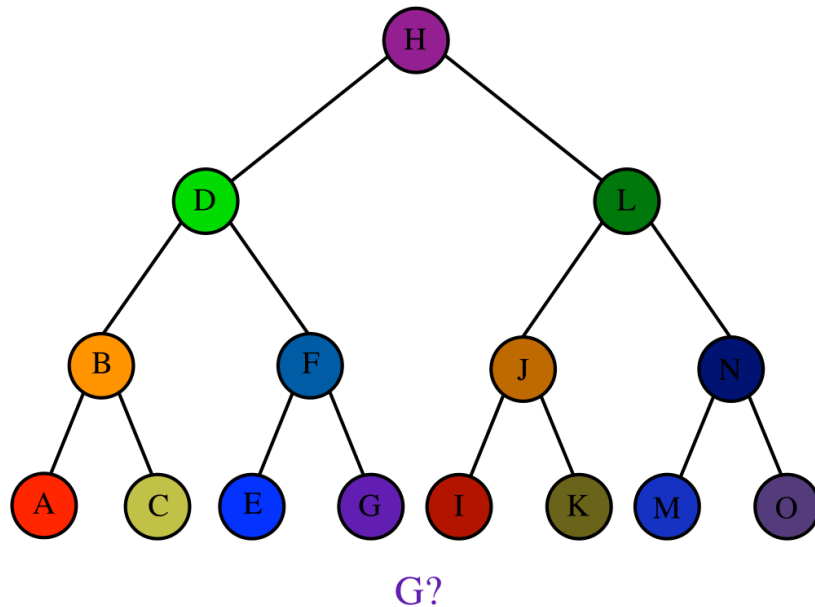
Wt. = 4
Value = 8



Maximum wt. = 13

การค้นปริภูมิสถานะ (State space searching)

- ปริภูมิสถานะ คือ คำตอบทั้งหมดที่เป็นไปได้ทั้งหมดของปัญหานั้น ๆ
- เราสามารถหาคำตอบที่เหมาะสมจากการค้นหาในปริภูมิสถานะ



ทบทวนการท่องต้นไม้

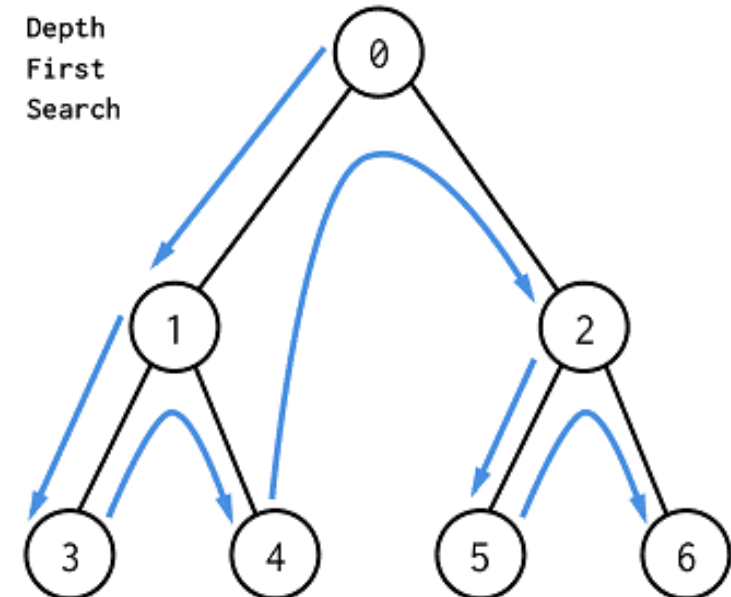
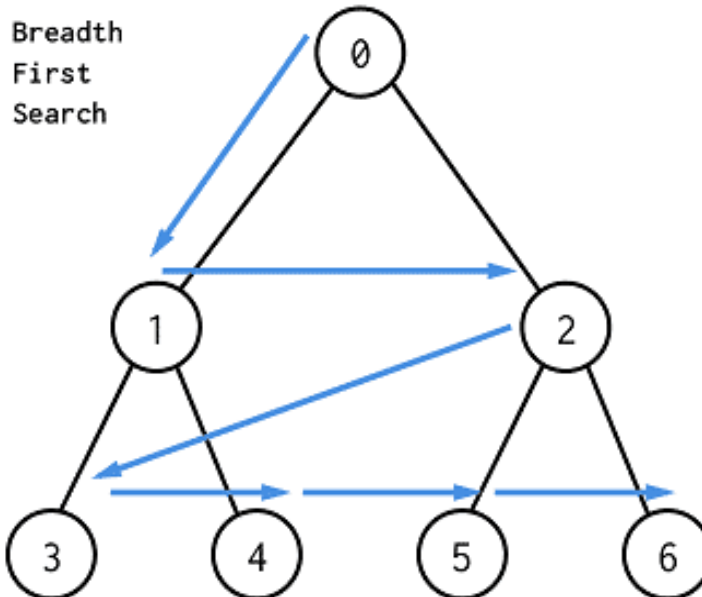
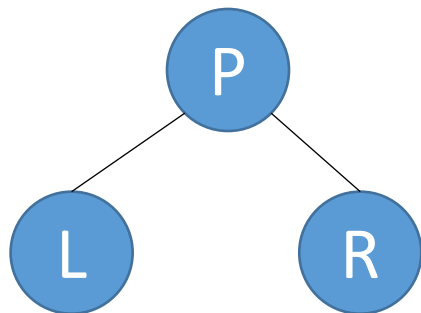
■ Breadth first search – Level-order

■ Depth first search

■ Pre-order (PLR)

■ In-order (LPR)

■ Post-order (LRP)



DFS vs BFS

```

1  #include<iostream>
2  #include<vector>
3  #include<queue>
4  #include<stack>
5  using namespace std;
6  //add the edge in graph
7  void edge(vector<int>adj[],int u,int v){
8      adj[u].push_back(v);
9  }

```

```

11 void bfs(int s,vector<int>adj[],bool visit[]){
12     queue<int>q;//queue in STL
13     q.push(s);
14     visit[s]=true;
15     while(!q.empty()){
16         int u=q.front();
17         cout<<u<<" ";
18         q.pop();
19         //loop for traverse
20         for(int i=0;i<adj[u].size();i++){
21             if(!visit[adj[u][i]]){
22                 q.push(adj[u][i]);
23                 visit[adj[u][i]]=true;
24             }
25         }
26     }
27 }

```

```

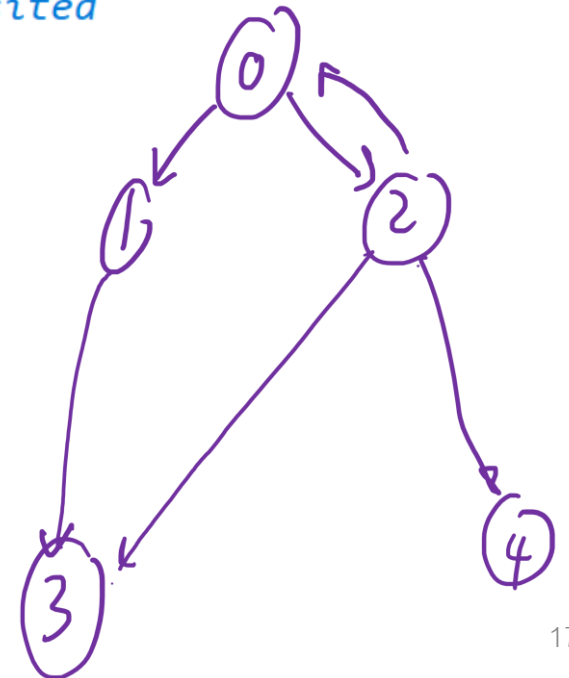
29 void dfs(int s,vector<int>adj[],bool visit[]){
30     stack<int>stk;//stack in STL
31     stk.push(s);
32     visit[s]=true;
33     while(!stk.empty()){
34         int u=stk.top();
35         cout<<u<<" ";
36         stk.pop();
37         //loop for traverse
38         for(int i=0;i<adj[u].size();i++){
39             if(!visit[adj[u][i]]){
40                 stk.push(adj[u][i]);
41                 visit[adj[u][i]]=true;
42             }
43         }
44     }
45 }

```

```

47 int main(){
48     vector<int>adj[5];//vector of array to store the graph
49     bool visit[5];//array to check visit or not of a node
50     //initially all node are unvisited
51     for(int i=0;i<5;i++){
52         visit[i]=false;
53     }
54     //input for edges
55     edge(adj,0,2);
56     edge(adj,0,1);
57     edge(adj,1,3);
58     edge(adj,2,0);
59     edge(adj,2,3);
60     edge(adj,2,4);

```



```
61  cout<<"BFS traversal is"<<"  ";
62  //call bfs funtion
63  bfs(0,adj,visit);//1 is a starting point
64  cout<<endl;
65  //again initialise all node unvisited for dfs
66  for(int i=0;i<5;i++){
67      visit[i]=false;
68  }
69  cout<<"DFS traversal is"<<"  ";
70  //call dfs function
71  dfs(0,adj,visit);//1 is a starting point
72  }
```

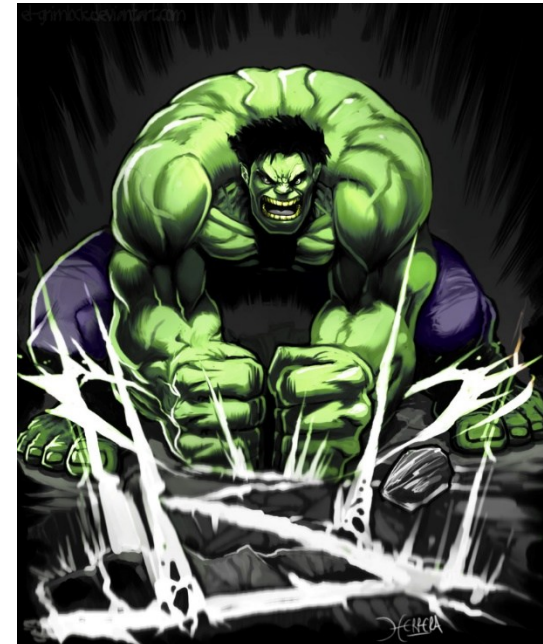


การค้นปริภูมิสถานะ (State space searching)

- เราต้องทำการสร้างปริภูมิสถานะเพื่อนำมาใช้ค้นหาคำตอบ ซึ่งสามารถทำได้ 2 วิธี
 - สร้างแบบไม่มีแบบแผน (Non-systematic)
 - สร้างแบบมีแบบแผน (Systematic) – การเรียงสับเปลี่ยน และการจัดหมู่
- การค้นปริภูมิสถานะ สามารถทำได้ 2 วิธี
 - การค้นหาจนหมดแรง (Exhaustive searching) หรือ การตะลุยคำตอบ (Brute-force)
 - การย้อนรอย (Backtracking)

การตะลุยคำตอบ (Brute-force)

- เป็นการแก้ปัญหาอย่างตรงไปตรงมา เข้าใจง่าย
- โดยทำการตรวจสอบทุกเซตคำตอบเพื่อหาเซตคำตอบที่เหมาะสมที่สุด ตามเงื่อนไขของปัญหา (Constraint Satisfaction)
- เหมาะกับปัญหขนาดเล็กที่มีจำนวน input ไม่มาก
- ตัวอย่างเช่น
 - Bubble sort, Selection sort, Insertion sort
 - Sequential search
 - DFS และ BFS



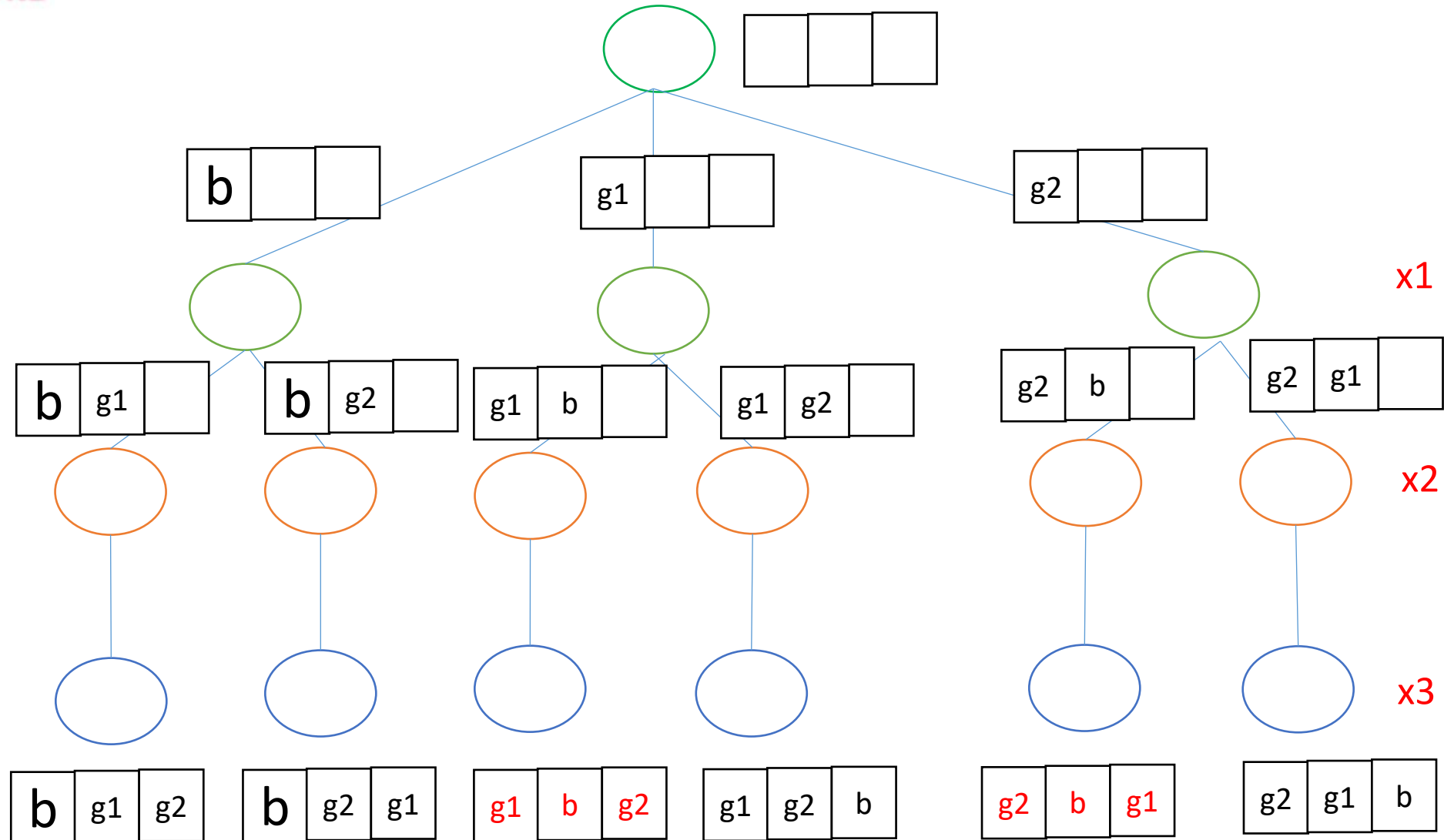
Exhaustive Search

- หลักการที่ใช้ในการค้นหาคำตอบของปัญหาเหล่านี้ ก็คือการสำรวจคำตอบที่เป็นไปได้ทั้งหมด เรียกว่า Brute force หรือ blind search จากนั้นนำคำตอบทั้งหมดที่สำรวจได้มาทดสอบกับเงื่อนไขบังคับของปัญหา เพื่อเลือกเฉพาะคำตอบที่ต้องการ
- อัลกอริทึม Exhaustive search มี 2 ขั้นตอนหลัก ได้แก่
 1. การเข้ารหัสคำตอบในรูปของเซต (set)
 2. วิธีสำรวจคำตอบอย่างมีระบบ (systematic)

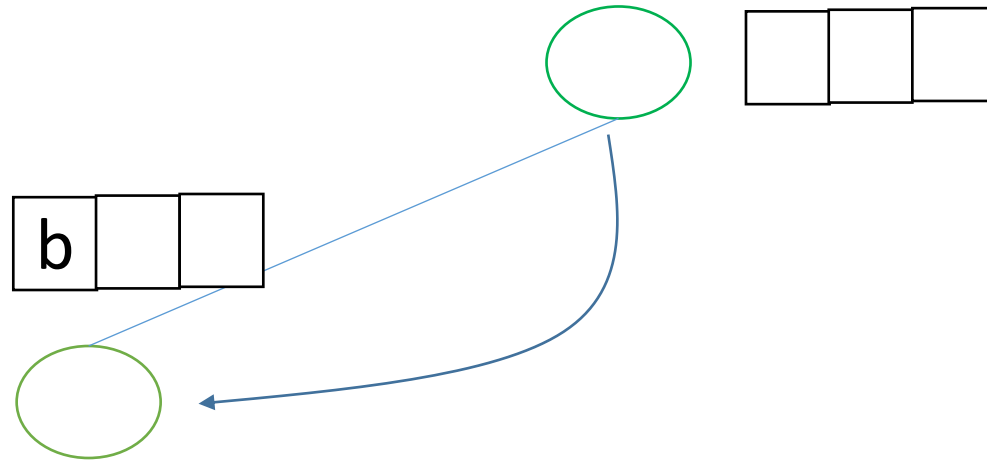
Exhaustive search (2)

- มักนิยามคำตอบในรูปของเซต (set) ตัวอย่างเช่น
 - การแสดงคำตอบของปัญหาจัดเก็้อ อาจนิยามเซตคำตอบ $X = \{x1, x2, x3\}$ โดยลำดับ $x1, x2$, และ $x3$ จะแทนเก็้อแต่ละตัว และค่าของ $x1, x2, x3$ แทนสถานะของเก็้อ เช่น $X = \{'boy', 'girl1', 'girl2'\}$
- ขั้นตอนถัดไปคือการสำรวจสถานะของเก็้อที่เป็นไปได้ใน X โดยการสร้างต้นไม้คำตอบ (solution tree) เพื่อสับเปลี่ยนลำดับของเด็กแต่ละคน
- อย่างไรก็ตามการสร้างต้นไม้คำตอบที่สมบูรณ์แบบ (complete tree) ล่วงหน้าอาจไม่ใช่วิธีที่เหมาะสมเนื่องจากต้นไม้คำตอบอาจมีขนาดใหญ่จนเกินไป

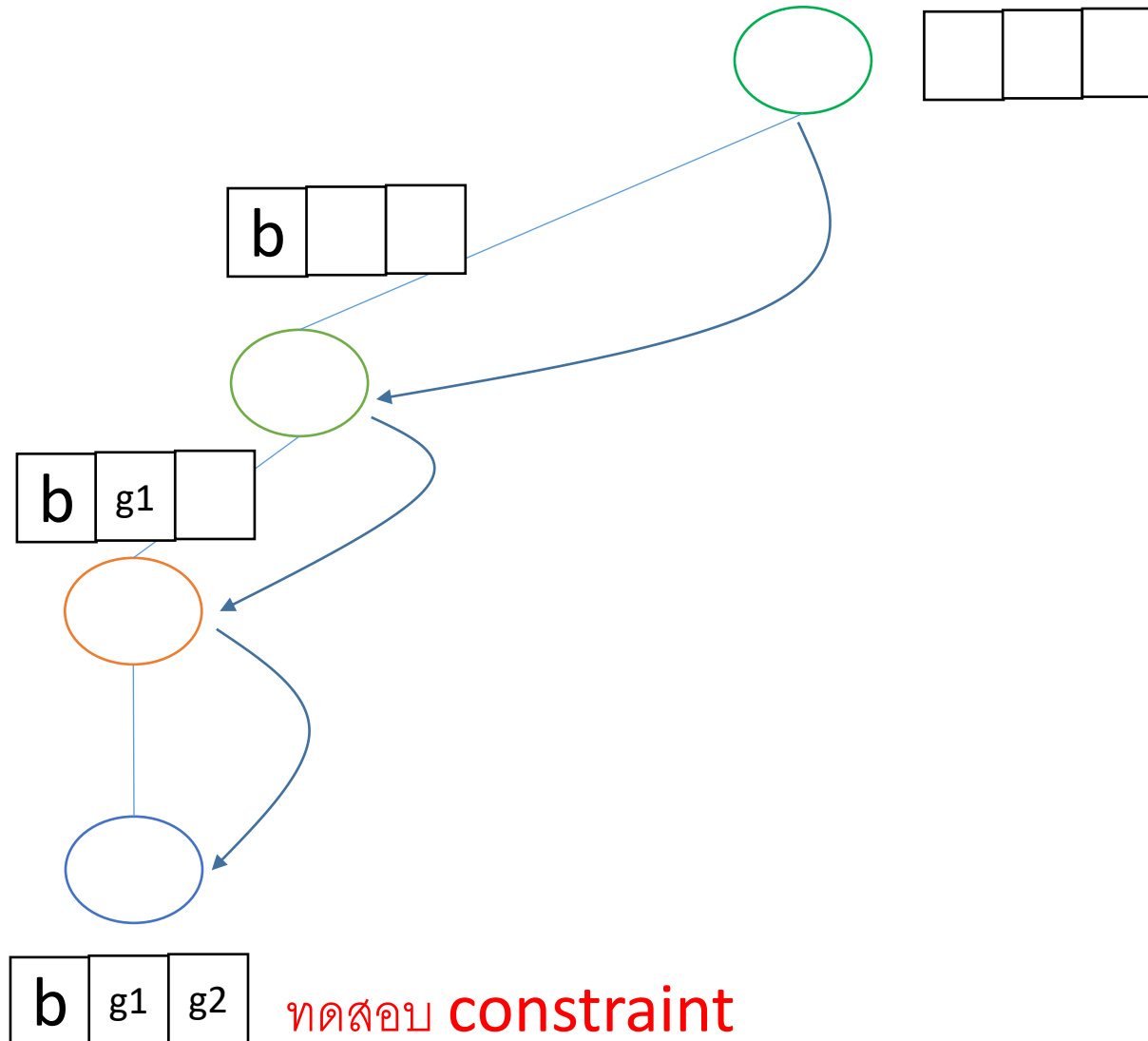
ต้นไม้คำตอบที่สมบูรณ์แบบ



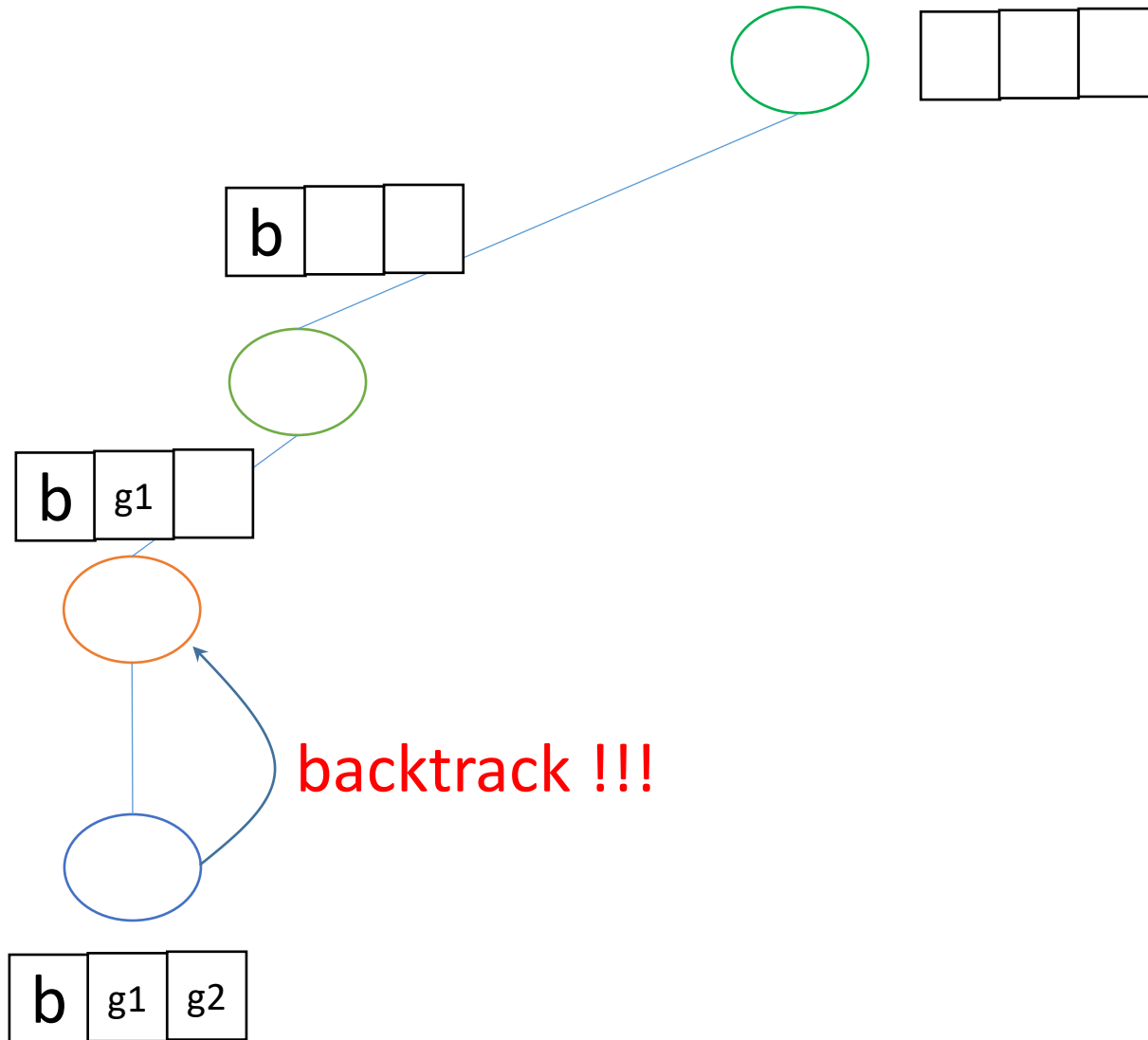
การสำรวจคำตอบด้วยวิธี Exhaustive Search



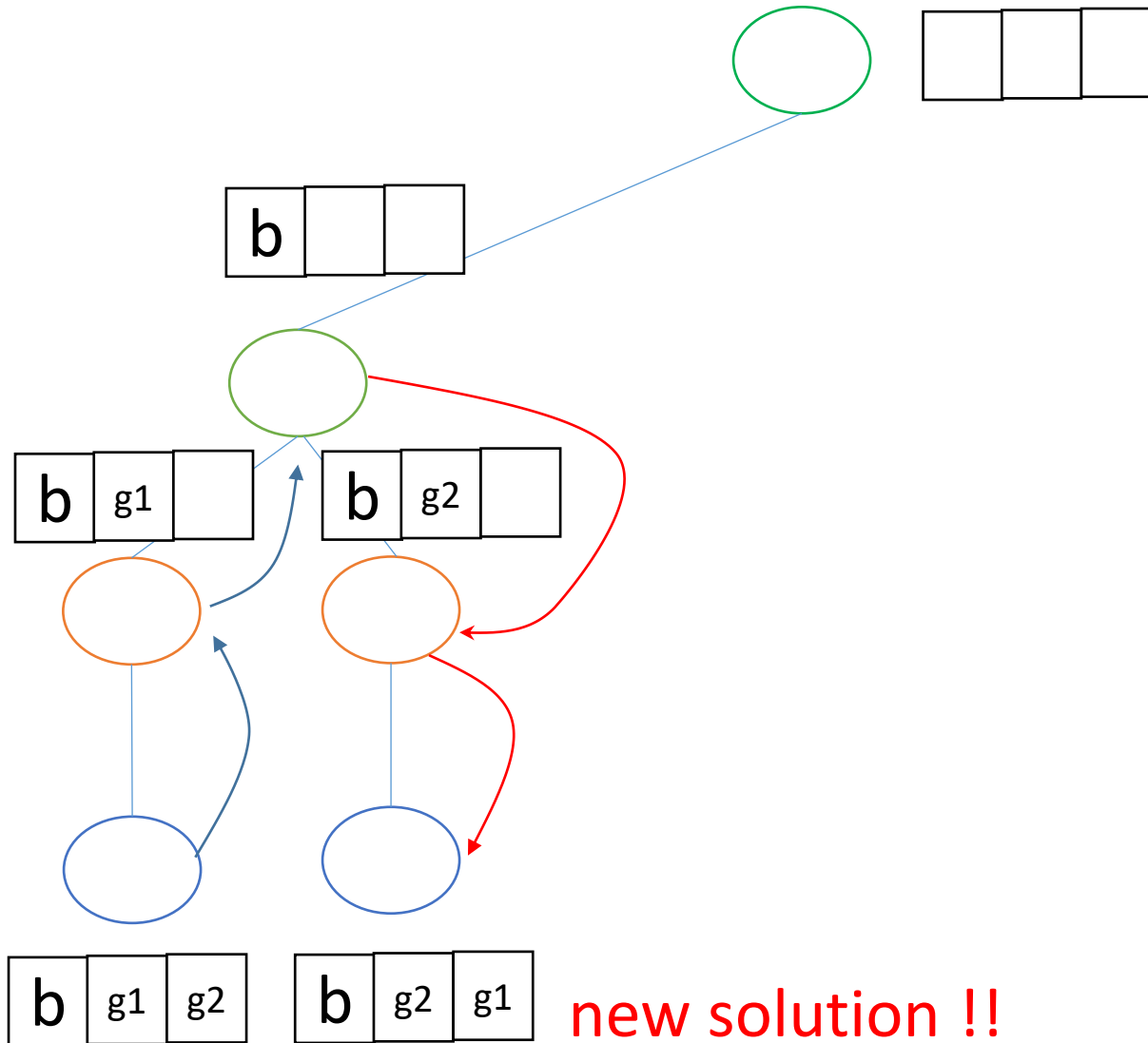
การสำรวจคำตอบด้วยวิธี Exhaustive Search



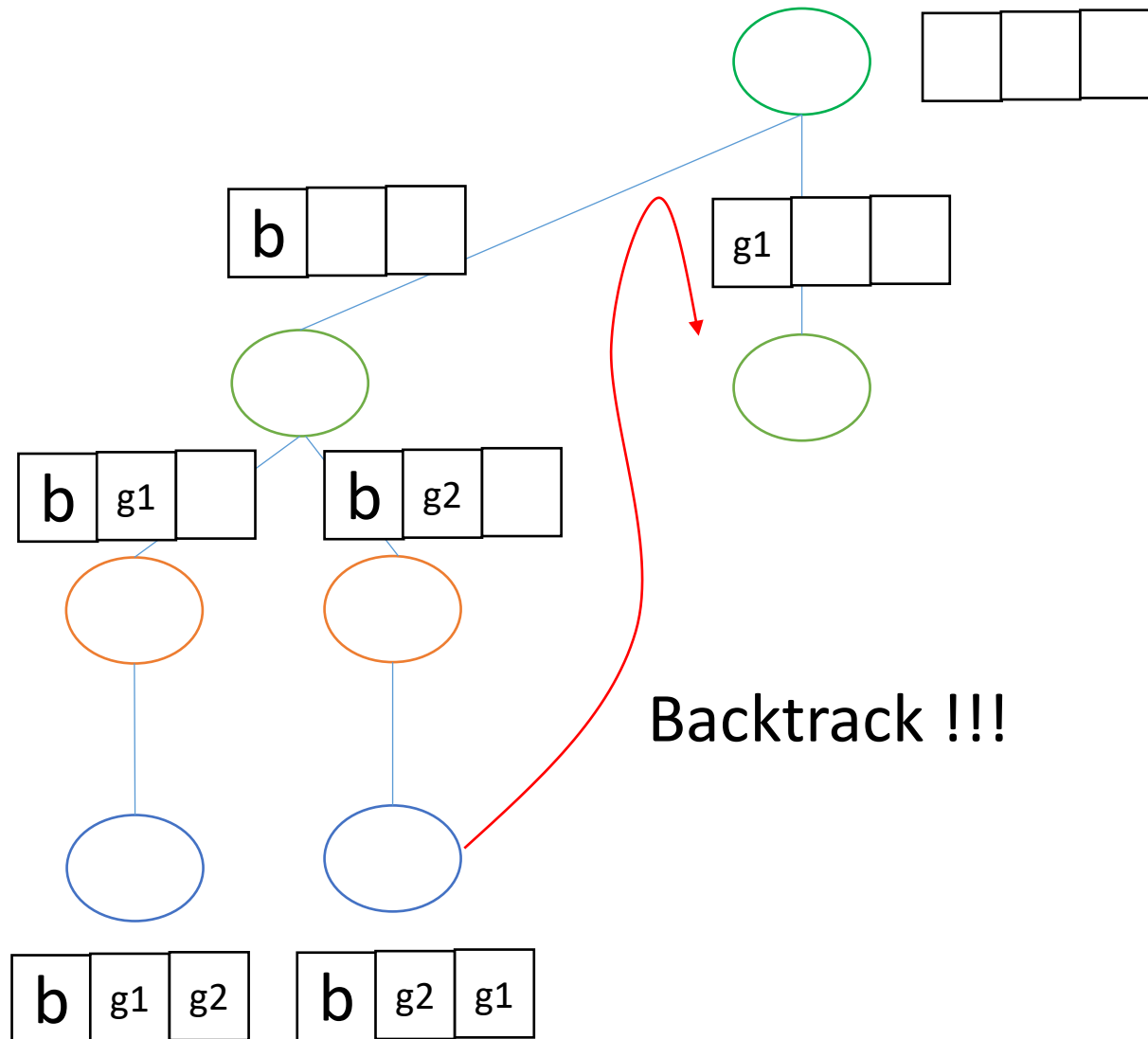
การสำรวจคำตอบด้วยวิธี Exhaustive Search



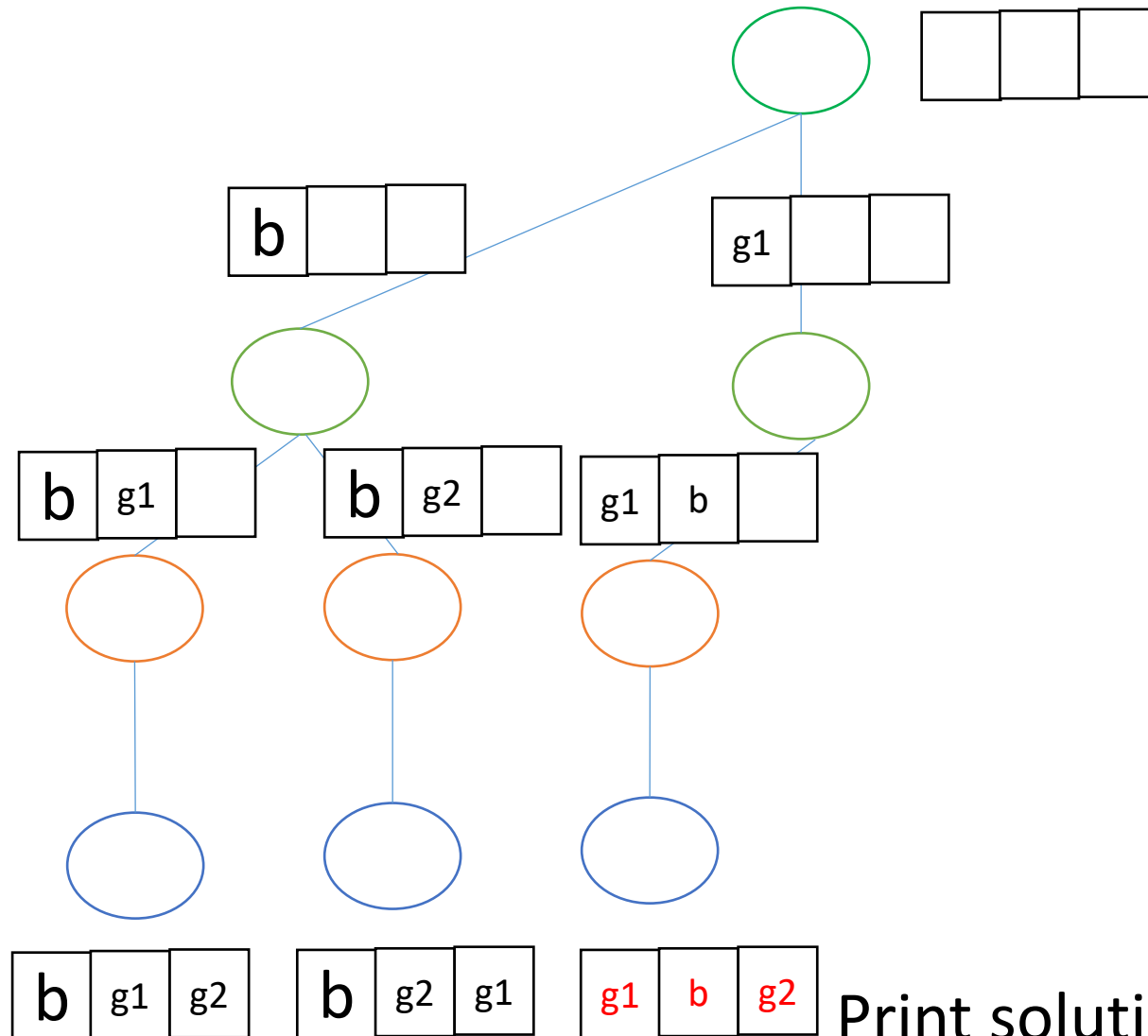
การสำรวจคำตอบด้วยวิธี Exhaustive Search



การสำรวจคำตอบด้วยวิธี Exhaustive Search

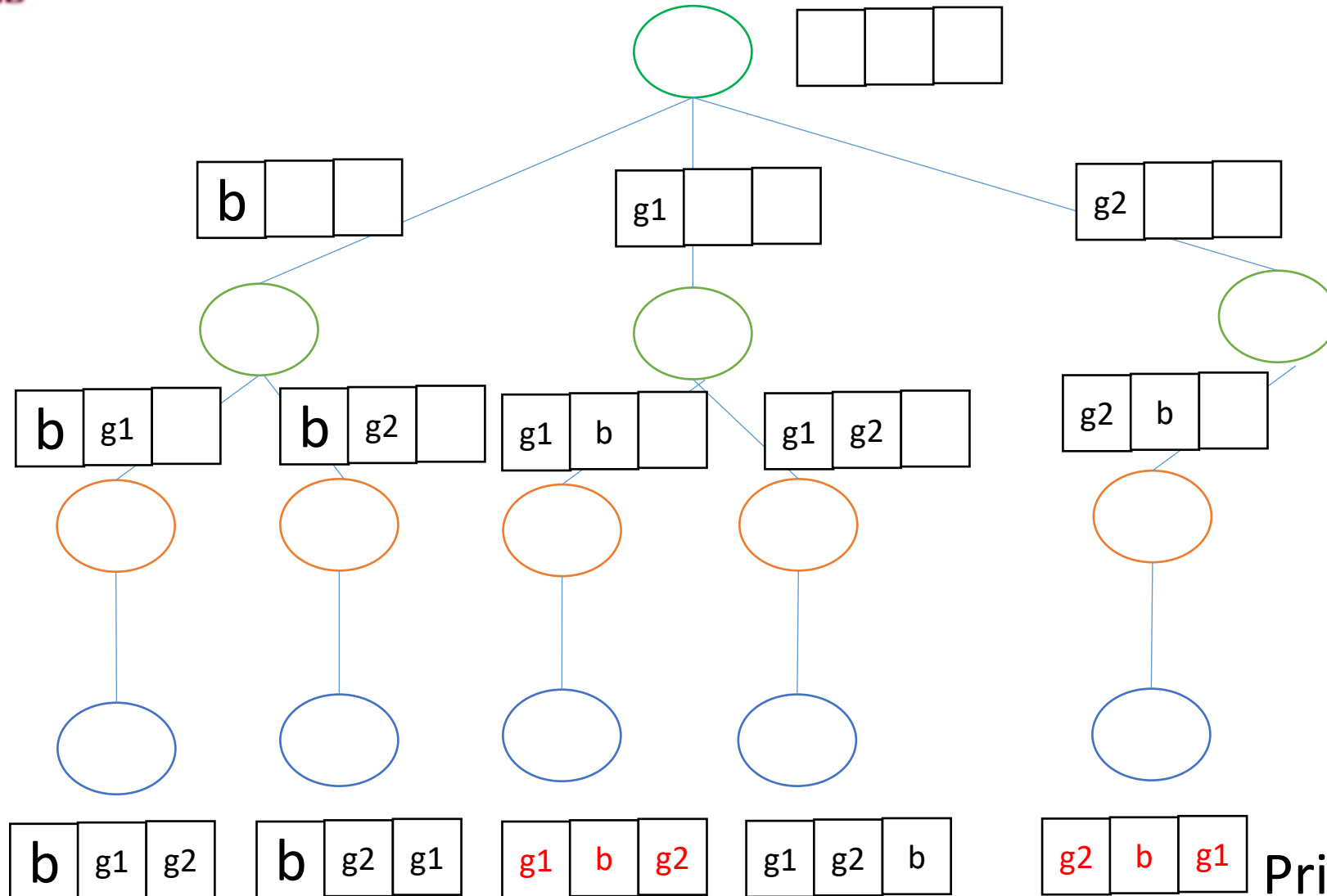


การสำรวจคำตอบด้วยวิธี Exhaustive Search



Print solution !!!

การสำรวจคำตอบด้วยวิธี Exhaustive Search



Print solution !!!





Recursive tree for Permutation

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void permute(int X[], int start, int end) {
    if (start == end) {
        // Print the current permutation
        for (int i = 1; i <= end; i++) {
            printf("%d ", X[i]);
        }
        printf("\n");
        return;
    }

    for (int i = start; i <= end; i++) {
        swap(&X[start], &X[i]); // Swap elements
        permute(X, start + 1, end); // Recursively permute the remaining elements
        swap(&X[start], &X[i]); // Undo the swap
    }
}

int main() {
    int n = 3;
    int X[] = {-1, 1, 2, 3}; // 1 = boy, 2 = g1, 3 = g2, -1 = no use

    permute(X, 1, n);

    return 0;
}
```

1 2 3
1 3 2
2 1 3
2 3 1
3 2 1
3 1 2

Workshop 1

- จงปรับปรุงโค้ด Permutation ให้แสดงเฉพาะคำตอบที่ต้องการ

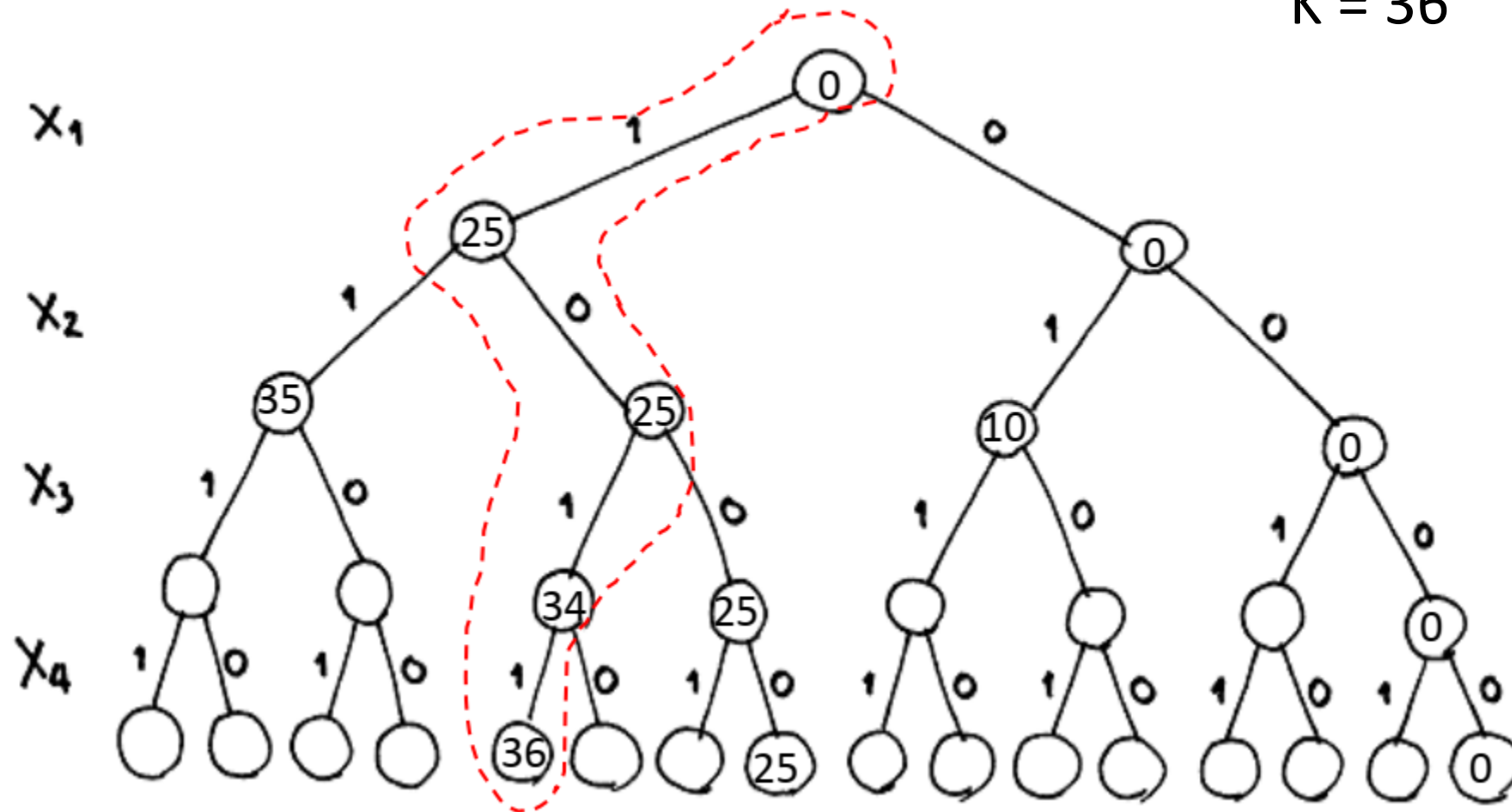


Subset Sum with Exhaustive Search

- กำหนดให้ $A = \{25, 10, 9, 2\}$ จงหาเซตย่อยที่ให้ผลรวมเท่ากับ 36
- นิยามคำตอบของปัญหานี้แทนด้วย $X = \{x_1, x_2, x_3, x_4\}$ โดยที่ x_i แทนสถานะของสมาชิกแต่ละตัวของ A และ ค่า $x_i = '1'$ หมายถึงสมาชิกตำแหน่ง i ถูกเลือก (include) และ ค่า $x_i = '0'$ หมายถึงสมาชิกในตำแหน่งที่ i ไม่ถูกเลือก (exclude)
- ตัวอย่างเช่น $\{25, 9\}$ แทนด้วยเซตคำตอบ $X = \{1, 0, 1, 0\}$ และ $\{10, 9, 2\}$ แทนด้วยเซตคำตอบ $X = \{0, 1, 1, 1\}$ เป็นต้น

Solution tree of subset sum (แบบที่ 1)

$A = \{25, 10, 9, 2\}$
 $K = 36$





```
#include <stdio.h>
```

```
void print_sol(int x[], int n) {
    for (int i = 1; i <= n; i++) {
        printf("%d ", x[i]);
    }
    printf("\n");
}

void subset1(int x[], int l, int r) {
    if (l == r) {
        print_sol(x, r);
    } else {
        x[l + 1] = 1;
        subset1(x, l + 1, r);
        x[l + 1] = 0;
        subset1(x, l + 1, r);
    }
}

int main() {
    int n = 3; // Number of elements
    int x[n + 1]; // Array to store subset

    subset1(x, 0, n);
    return 0;
}
```

Subset1

(generate binary codes)



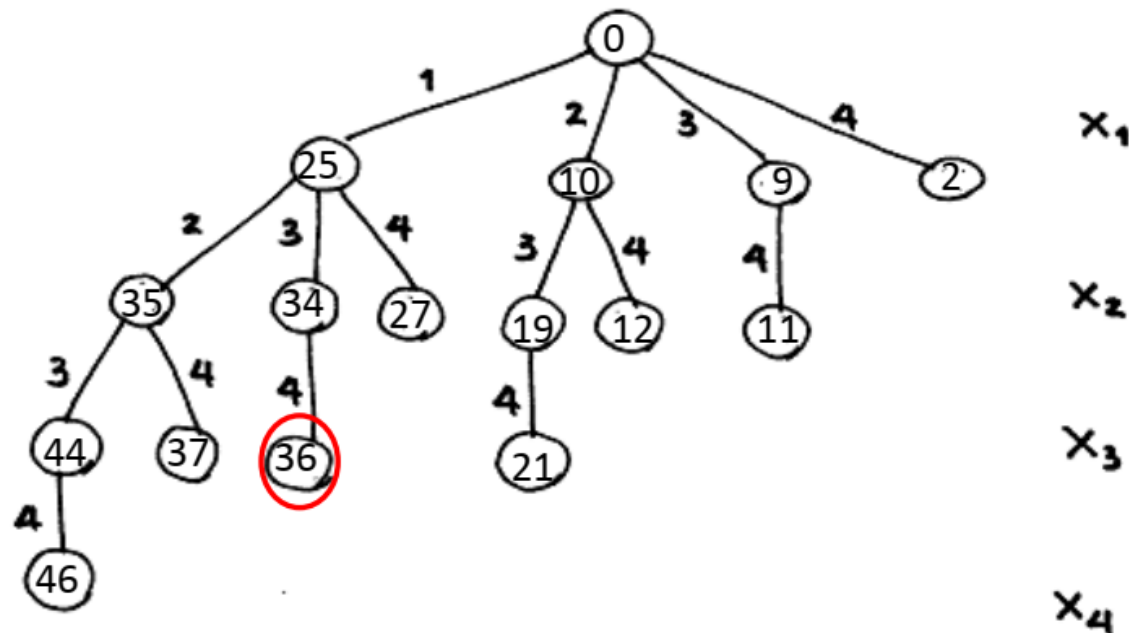
1	1	1
1	1	0
1	0	1
1	0	0
0	1	1
0	1	0
0	0	1
0	0	0

Workshop 2

- จงปรับปรุงโค้ด subset1 สำหรับการหาผลรวมของเซตย่อยใน $A = \{25, 10, 9, 2\}$ ที่มีค่าเท่ากับ 36

Solution tree of subset sum (แบบที่ 2)

- คำตอบของปัญหา Subset Sum อาจถูกนิยามในรูปของลำดับ (index) ของสมาชิกใน A เช่น กำหนดให้ $A = \{25, 10, 9, 2\}$, $k = 36$
 - $X = \{1, 2, 4\}$ แทน $\{25, 10, 2\} = 25 + 10 + 2 = 37$ หรือ
 - $X = \{2, 3\}$ แทน $\{10, 9\} = 10 + 9 = 19$





```
#include <stdio.h>
```

```
void print_sol(int x[], int n) {  
    for (int i = 1; i <= n; i++) {  
        printf("%d ", x[i]);  
    }  
    printf("\n");  
}
```

```
void subset2(int x[], int l, int r, int n) {  
    print_sol(x, l);  
    int j = (l == 0) ? 1 : x[l] + 1;  
    for (int i = j; i <= n; i++) {  
        x[l + 1] = i;  
        subset2(x, l + 1, r, n);  
    }  
}
```

```
int main() {  
    int n = 3; // Number of elements  
    int x[n + 1]; // Array to store subset  
  
    subset2(x, 0, n, n);  
  
    return 0;  
}
```

Subset2 (Generate sequences of indexes)

```
1  
1 2  
1 2 3  
1 3  
2  
2 3  
3  
|
```

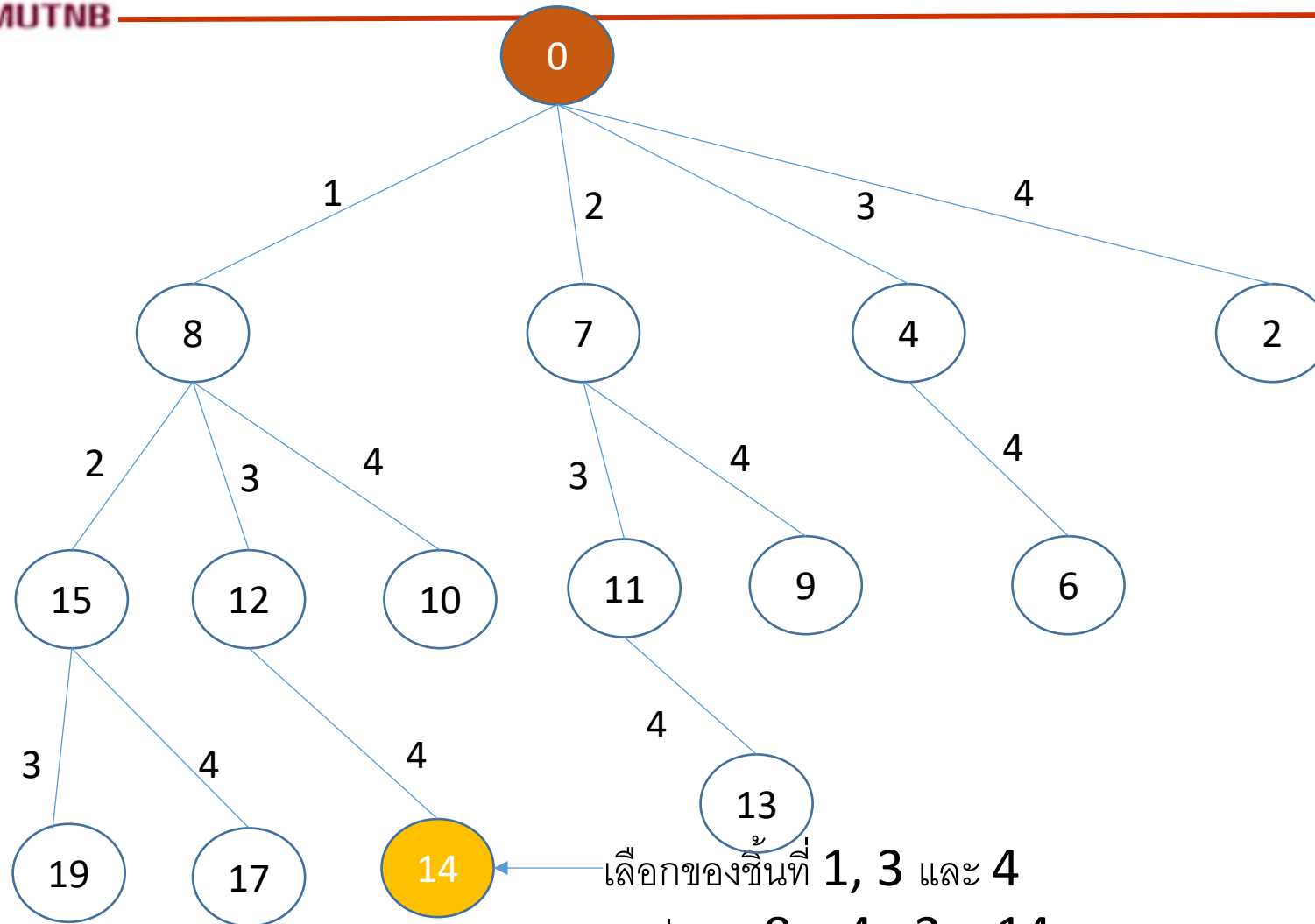
Knapsack problem

- สำหรับไบนารีเซต อาจแสดงคำตอบเป็นสถานะการเลือก/ไม่เลือกของแต่ละชั้น $\{x_1, x_2, \dots, x_n\}$ โดยที่ $x_i \in \{0, 1\}$ เช่น $\{1, 0, 1, 0, 1, 0\}$ หมายถึงใส่ของชั้นที่ 1, 3 และ 4 ลงไปในเป้ซึ่งมีน้ำหนักรวมคือ $10+7+6 = 23$ และมูลค่ารวมคือ $10+6+2 = 18$
- หรือแสดงคำตอบเป็นเซตย่อยโดยการเก็บลำดับของ object แต่ละชั้นที่ถูกเลือกใส่ในเป้

Object	w_i	v_i
O1	12	8
O2	5	7
O3	4	4
O4	2	2
Σ	23	21

$$W = 18$$

Solution tree of Knapsack problem



$W = 18$

Object	w_i	v_i
O1	12	8
O2	5	7
O3	4	4
O4	2	2
Σ	23	21



ทดลองวาด Solution tree

แบบ binary code สำหรับ knapsack problem

$W = 18$

Object	w_i	v_i
O1	12	8
O2	5	7
O3	4	4
O4	2	2
Σ	23	21

การเข้ารหัสคำตอบของ N queens

	0	1	2	3
0			♔	
1	♔			
2				♔
3		♔		

queen	1	2	3	4
row	1	3	0	2
col	0	1	2	3

แบบที่ 1

ต้นไม้คำตอบจะมีขนาดใหญ่มาก เนื่องจากมีแถวและคอลัมน์
ต้องทำการ generate ค่อนข้างมาก

col	0	1	2	3
row	1	3	0	2

แบบที่ 2

ต้นไม้คำตอบจะกระชับและใช้ space ลดลง

การตรวจสอบคำตอบใน N queens

■ ทดสอบเงื่อนไขการวางตำแหน่งของควีนแต่ละตัวบนบอร์ด ดังนี้

1). $|X.col - Y.col| = |X.row - Y.col|$ หรือ

2). $X.row \neq Y.row$

	0	1	2	3
0			♔	
1	♔			
2				♔
3		♔		

Q1: (0, 1) และ Q2: (1, 3)

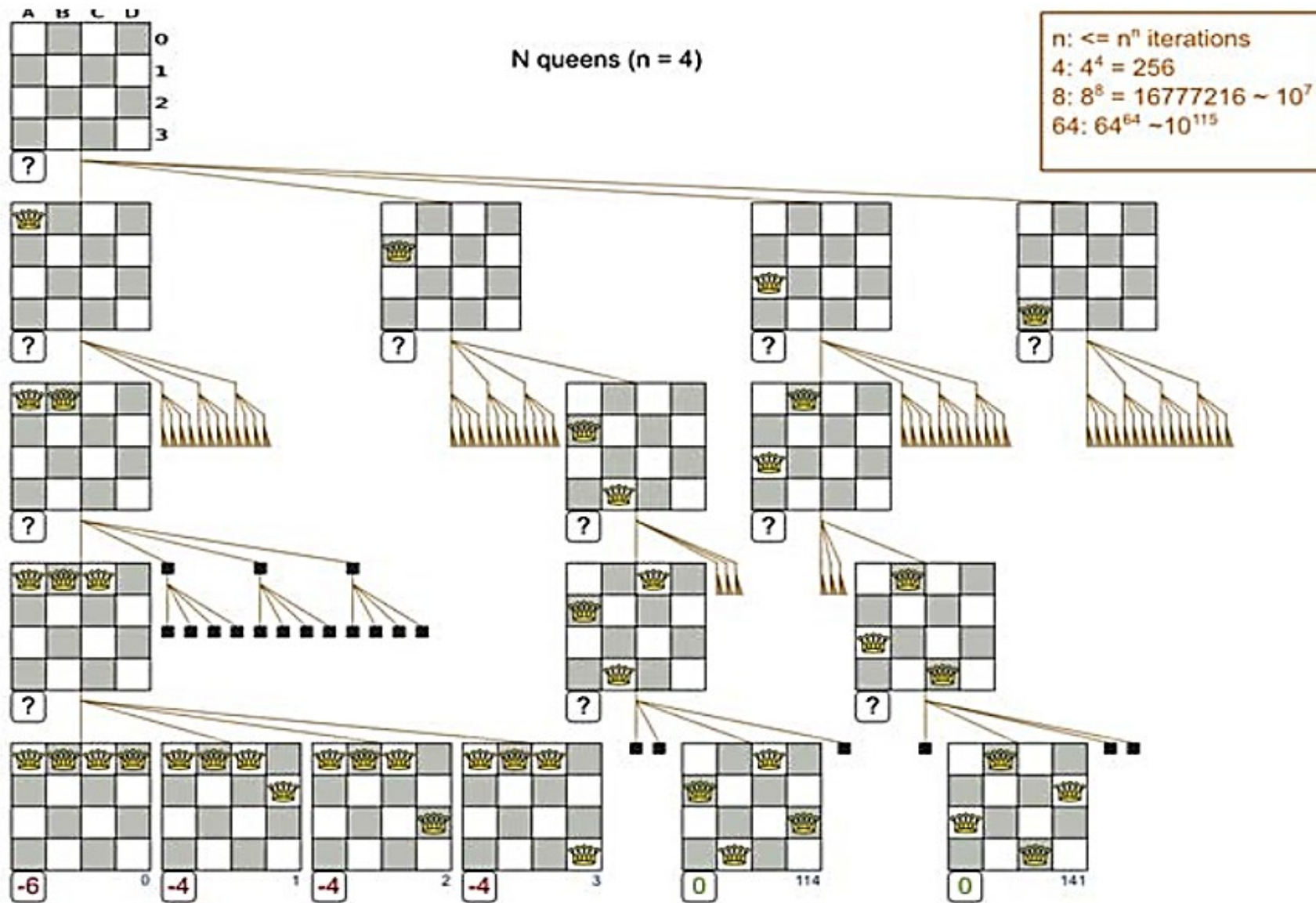
$$|0 - 1| < |1 - 3|$$

	A	B	C	D
0		♔		♔
1	♔			
2			♔	
3				

Q1: (0, 1) และ Q2: (1, 0)

$$|0 - 1| = |1 - 0|$$

Board tree (แบบที่ 1)



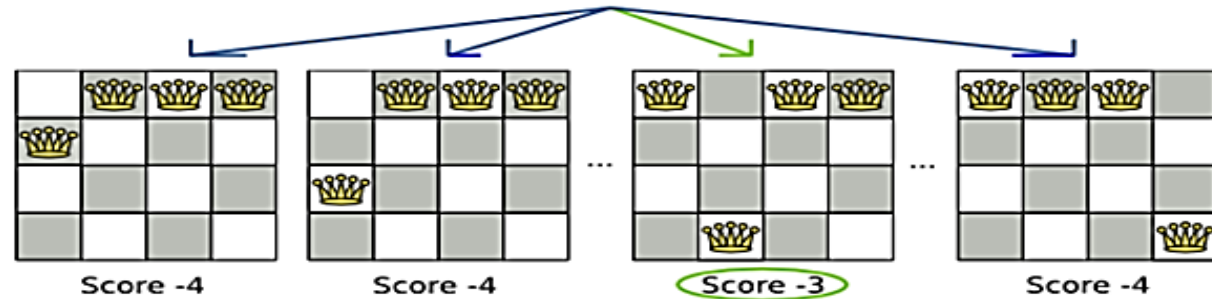
Solution tree of 4 queens ของบอร์ดแบบที่ 2

Step 0

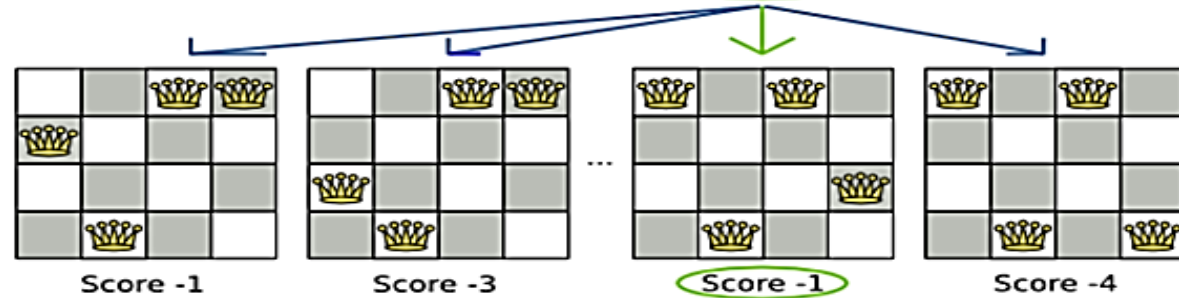
	A	B	C	D
0				
1				
2				
3				

Score -6

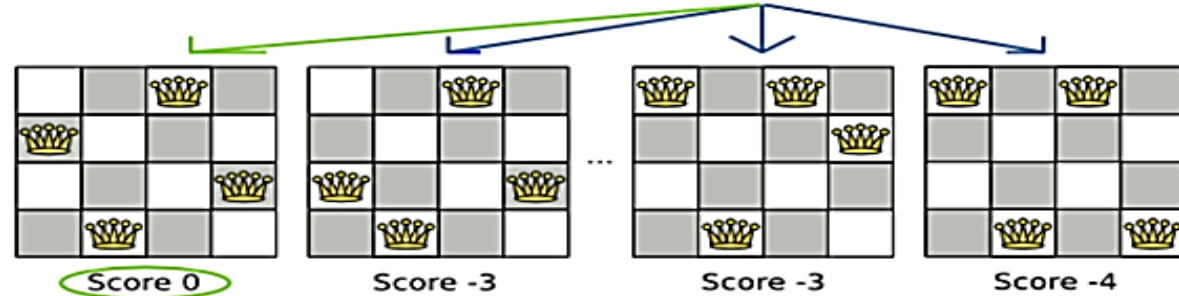
Step 1



Step 2



Step 3



```
#include<stdio.h>

int n=4, x[4];

void n_queens(int k) {
    if (k == n) {          // place all queens on the board
        int flag = 0;

        for(int j = 1; j < k ; j++)    // check any two queens
        {   for(int p = 0 ; p < j ;p++)
            {   if(abs(p-j) == abs(x[p]-x[j]) ||
                (x[j] == x[p]))
                flag = 1;
            }
        }

        if(flag == 0)      // no conflict print sol.
        {   for(int i=0; i<k; i++)
            printf("%d ", x[i]);
            printf("\n");
        }
    } else {
        for (int row = 0; row < n; row++) {    // place queen k on row i
            x[k] = row;
            n_queens(k + 1);    // place next queen
        }
    }
}

int main() {
    n_queens(0);
    return 0;
}
```

4 Queens with Exhaustive Search

1	3	0	2
2	0	3	1

อัลกอริทึมการย้อนรอย (Backtracking algorithm)

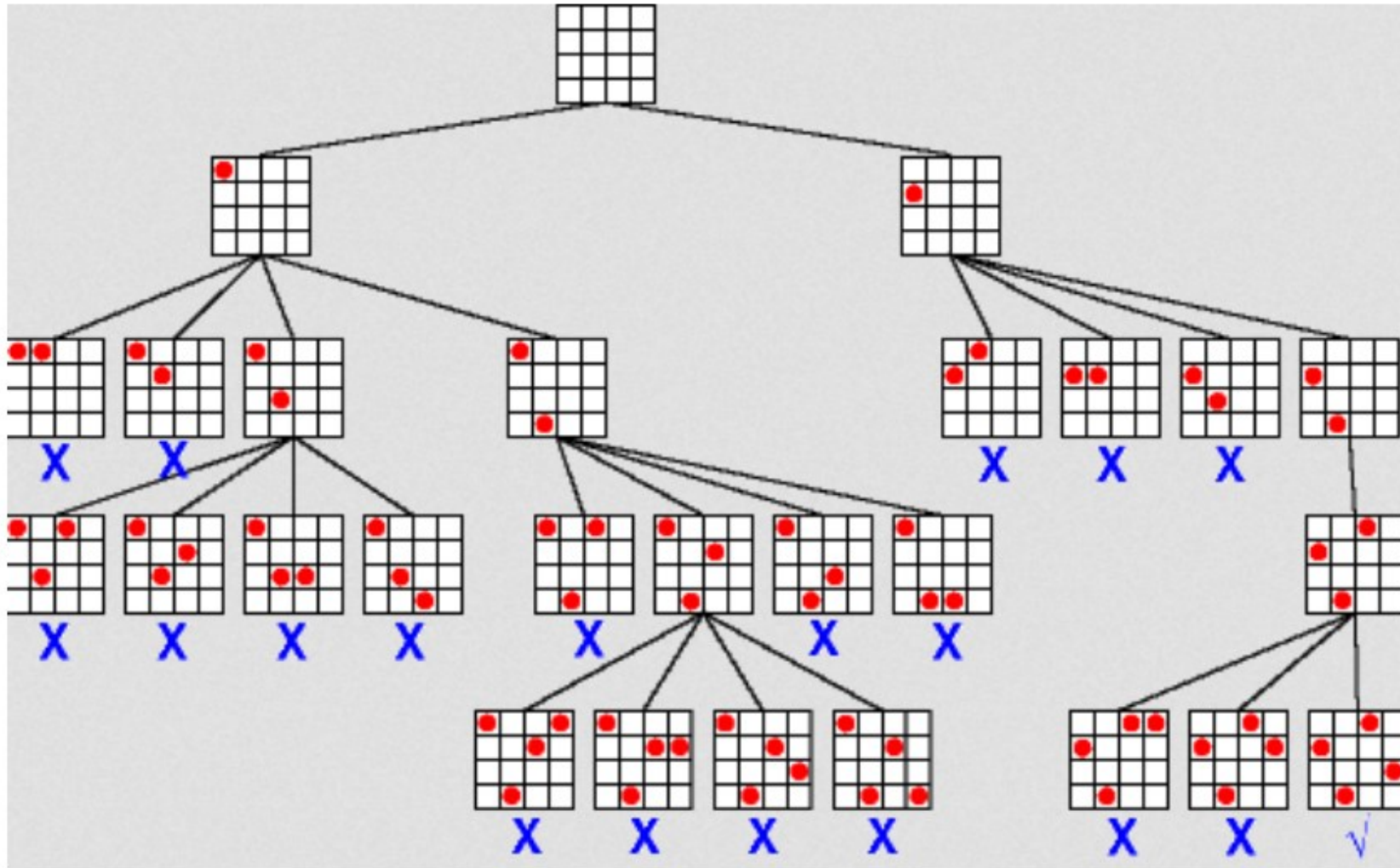
- ทำการตรวจสอบเฉพาะคำตอบที่อยู่ภายใต้เงื่อนไขของปัญหา
- โดยการสร้างชุดคำตอบทีละส่วน และพิจารณาว่าอยู่ภายใต้เงื่อนไขของปัญหาหรือไม่
- หากยังอยู่ภายใต้เงื่อนไข จะทำการสร้างส่วนของคำตอบถัดไป และทำการตรวจสอบใหม่
- หากไม่ จะทำการย้อนกลับ ไปยังชุดคำตอบก่อนหน้า และสร้างชุดคำตอบใหม่



อัลกอริทึมย้อนรอย (Backtracking algorithm)

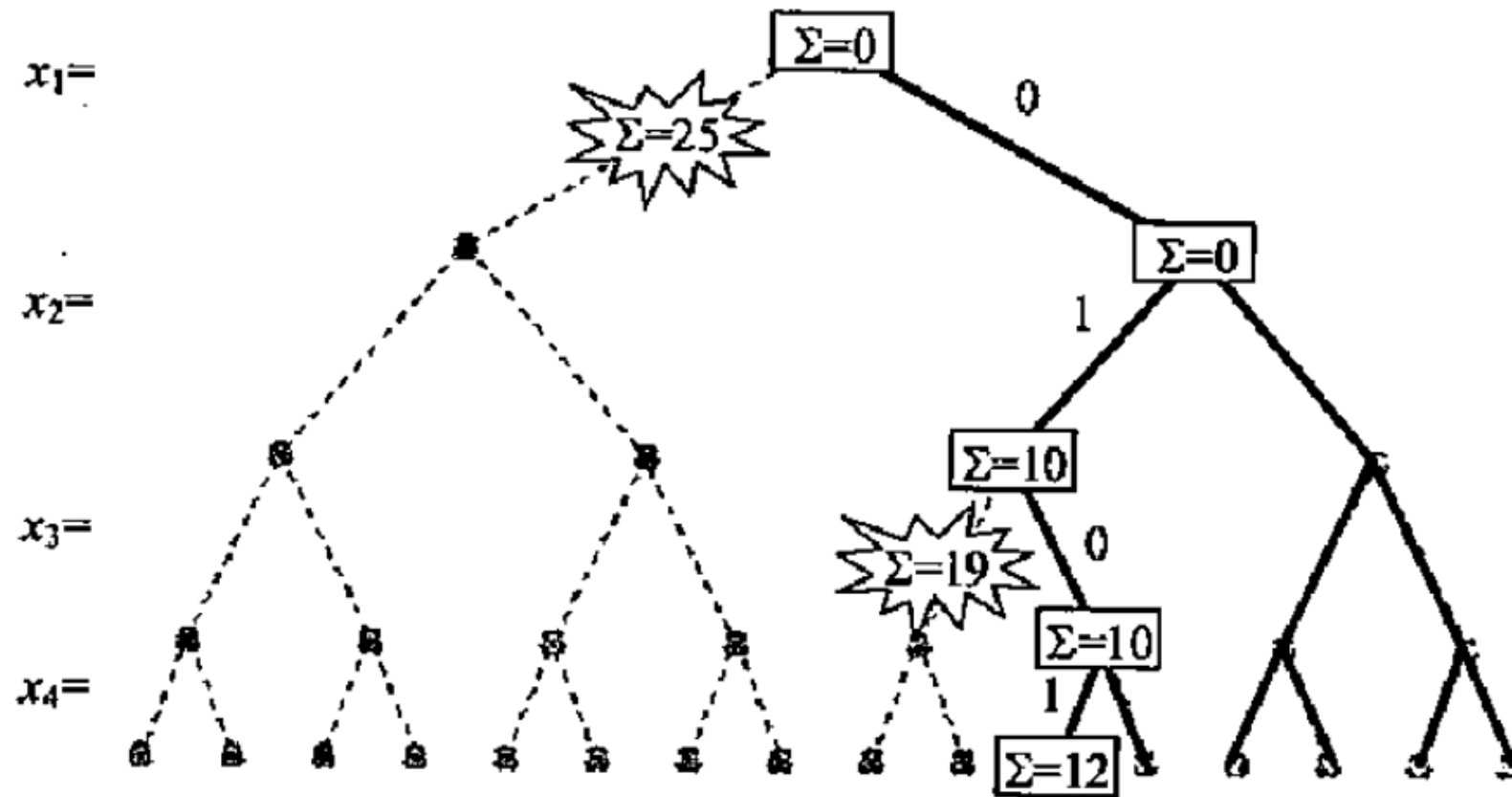
- เทคนิคช่วยเพิ่มประสิทธิภาพให้กับ Exhaustive Search เพื่อลดจำนวน search search ส่งผลให้ค้นหาคำตอบได้เร็วขึ้น
- แนวคิดของอัลกอริทึมย้อนรอยคือการสำรวจสถานะคำตอบที่ละเส้นทาง (path) และตัดเส้นทาง (pruning) ที่ไม่อาจนำไปสู่คำตอบของปัญหาทั้ง โดยการตรวจสอบสถานะของคำตอบปัจจุบันว่าขัดแย้งกับเงื่อนไขบังคับของปัญหาหรือไม่

ตัวอย่าง 4 queens with backtracking



Subset sum with backtracking

สมมติว่า $A = \{25, 10, 9, 2\}$ หาเซตย่อยที่ผลรวมเท่ากับ $P = 12$



Backtracking Example

```
1 #include <stdio.h>
2 #define MAX_N 100
3 int n, s, a[MAX_N], x[MAX_N];
4 void print_solution() {
5     int i;
6     for (i = 0; i < n; i++) {
7         if (x[i]) {
8             printf("%d ", a[i]);
9         }
10    }
11    printf("\n");
12 }
```

```
13 void backtrack(int i, int sum) {
14     int j;
15     if (i == n) {
16         if (sum == s)
17             print_solution();
18     }
19     else {
20         for (j = 0; j <= 1; j++) {
21             x[i] = j;
22             backtrack(i + 1, sum + j * a[i]);
23         }
24     }
25 }
26 }
```

```
27 int main() {
28     a[0] = 25; a[1] = 10; a[2] = 9; a[3] = 2;
29     n = 4;
30     s = 12;
31     backtrack(0, 0);
32     return 0;
33 }
```

Backtracking Example

■ Bounding function

```
void backtrack(int i, int sum) {  
    int j;  
    if (i == n) {  
        if (sum == s)  
            print_solution();  
    }  
    else { if(sum+a[i+1] <= s)           // bounding function  
        { for (j = 0; j <= 1; j++)  
            { x[i] = j;  
              backtrack(i + 1, sum + j * a[i]);  
            }  
        }  
    }  
}
```

```
void subset_sum(int x[], int l, int r, int sum) {
    if (l == r) {
        print_sol(x, r);
        return;
    }

    if (sum > target)                // pruning if subset sum is greater than target
        return;

    x[l + 1] = 1;                    // Include the current element
    sum += A[l];                     // compute subset sum
    subset_sum(x, l + 1, r, sum);

    x[l + 1] = 0;                    // Exclude the current element
    sum -= A[l];                     // compute subset sum
    subset_sum(x, l + 1, r, sum);
}

int main() {
    int n = 4;                       // Number of elements
    int x[n + 1];                     // Array to store subset

    subset_sum(x, 0, n, 0);

    return 0;
}
```

เพิ่มตัวแปร **sum** เพื่อทดสอบผลรวมของคำตอบปัจจุบันว่าเกินกว่าค่า **target** หรือไม่ ถ้าเกินก็จะทำการ **backtrack**



```
#include<stdio.h>

int n=4, x[4];

void n_queens(int k) {
    if (k == n) {          // place all queens on the board
        int flag = 0;

        for(int j = 1; j < k ; j++)    // check any two queens
        {   for(int p = 0 ; p < j ;p++)
            {   if(abs(p-j) == abs(x[p]-x[j]) ||
                (x[j] == x[p]))
                flag = 1;
            }
        }

        if(flag == 0)      // no conflict print sol.
        {   for(int i=0; i<k; i++)
            {   printf("%d ", x[i]);
                printf("\n");
            }
        }
    } else {
        for (int row = 0; row < n; row++) {    // place queen k on row i
            x[k] = row;
            n_queens(k + 1);    // place next queen
        }
    }
}

int main() {
    n_queens(0);
    return 0;
}
```

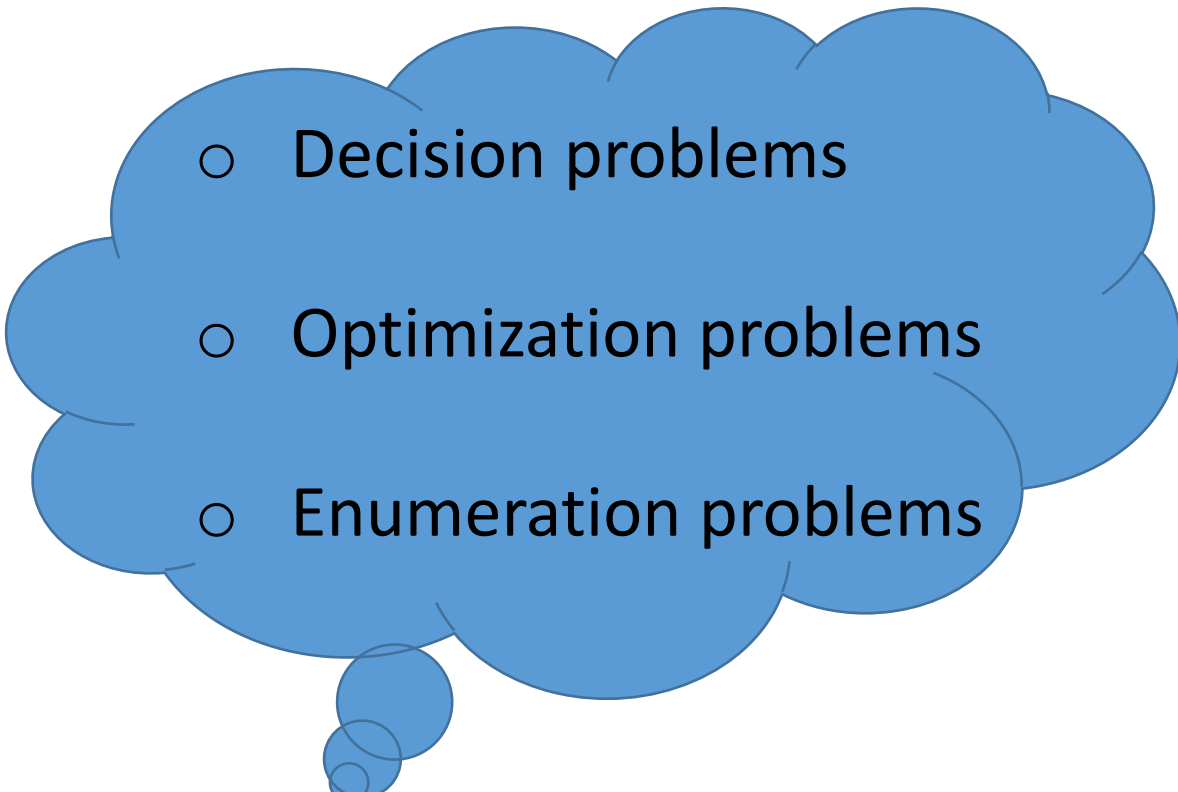
Workshop 3: 4 Queens with Backtracking

จงปรับปรุงโค้ด 4 queens โดยใช้
เทคนิค backtrack

1	3	0	2
2	0	3	1

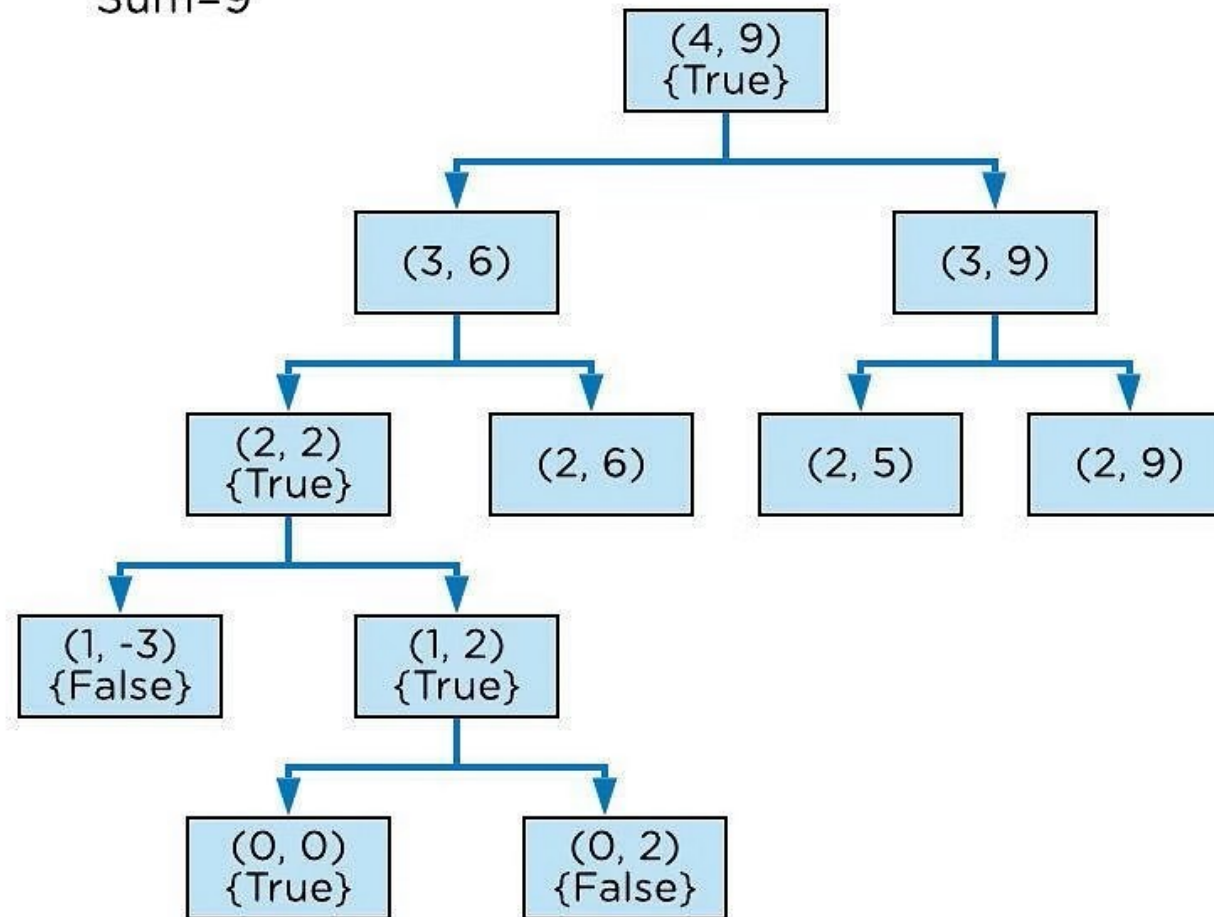
ตัวอย่างปัญหา

- Sum of subsets หรือ subset sum
- 0/1 Knapsack
- Graph coloring
- Job assignment
- Travelling Salesman
- Puzzles
 - Maze runner
 - N-Queens
 - Sudoku

- 
- Decision problems
 - Optimization problems
 - Enumeration problems

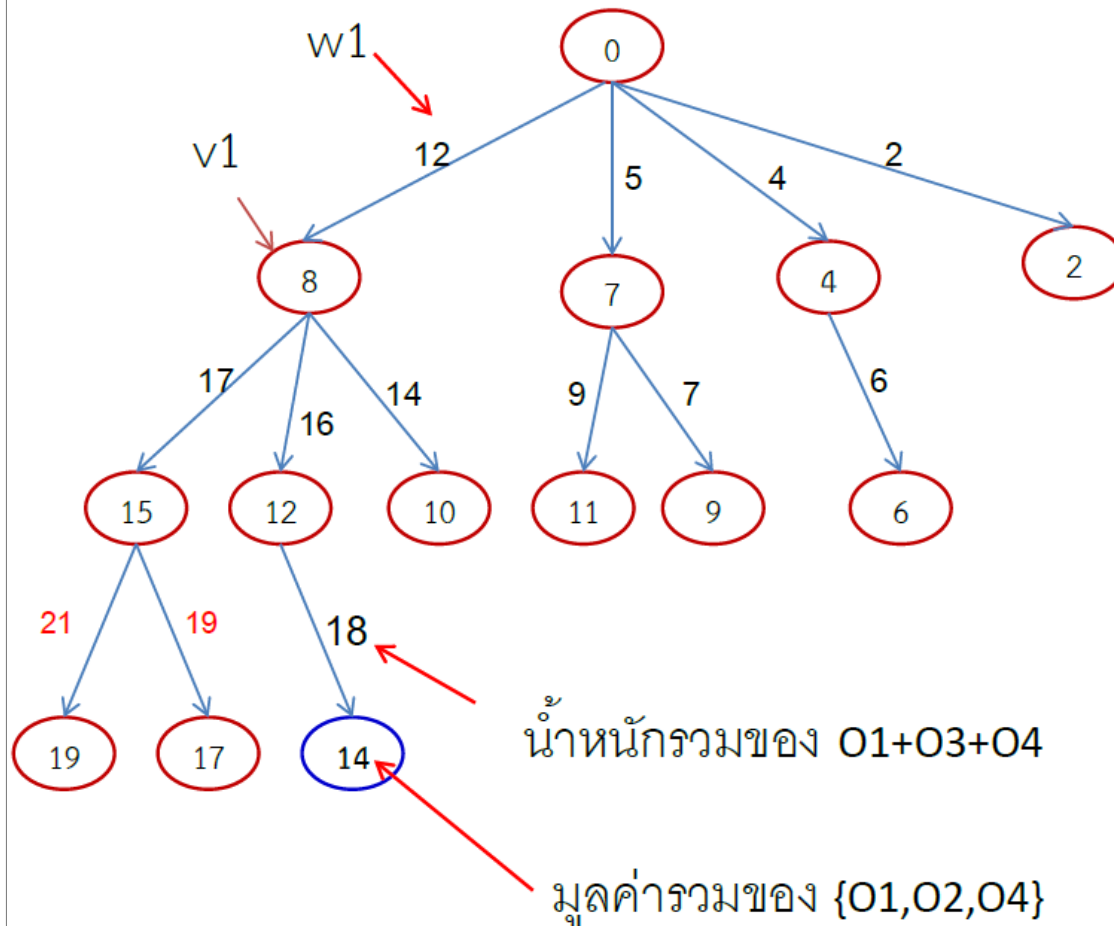
Sum of subsets

Set[] = {3, 4, 5, 2}
Sum = 9



0/1 Knapsack

Exhaustive Search for knapsack problem

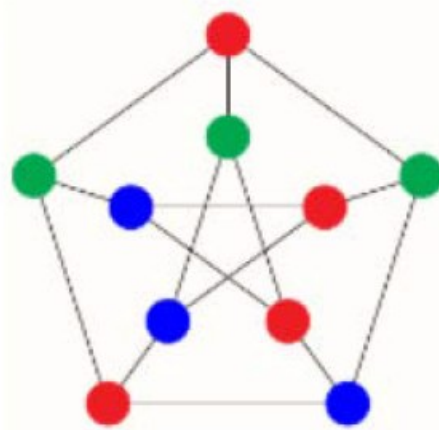
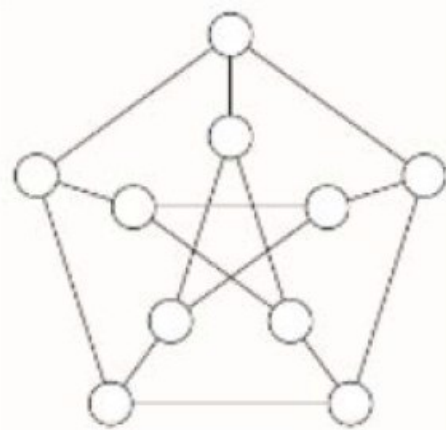


Object	w_i	v_i
O1	12	8
O2	5	7
O3	4	4
O4	2	2
Σ	23	21

$$W = 18$$

Graph coloring

กำหนดให้กราฟ G โหนด หากต้องการระบายสีโหนดบนกราฟ โดยมีเงื่อนไขว่า โหนดที่มีเอดจ์ร่วมกันจะต้องระบายสีแตกต่างกันจะมีวิธีที่เป็นไปได้ (มีหลายคำตอบ) หรืออาจถามว่าจำนวนสีน้อยที่สุด (มีคำตอบเดียว) ที่ใช้ระบายสีบนพื้นที่นี้ตามเงื่อนไข

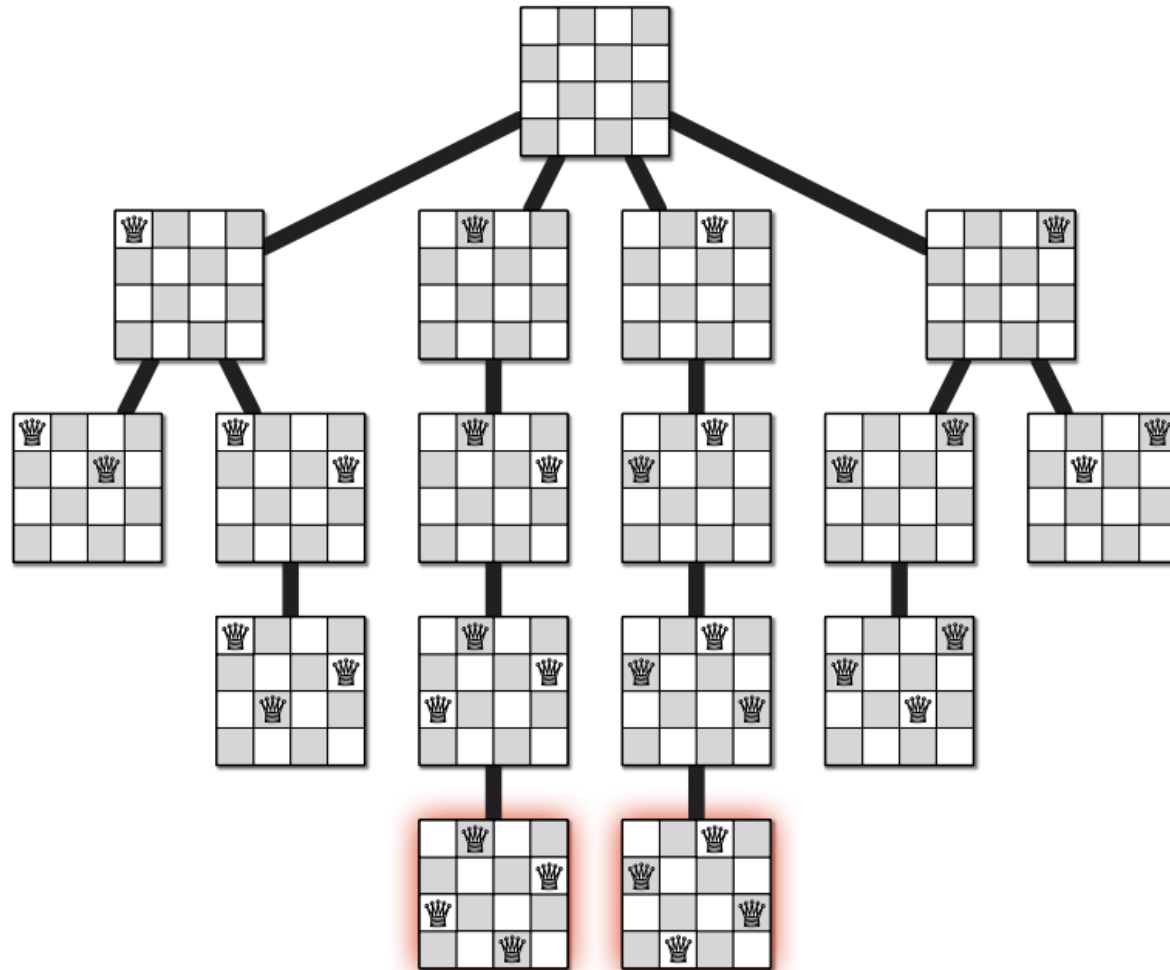


Graph coloring



Map coloring

N-Queens



```

1  #include <stdio.h>
2  #define MAX_N 100
3  int n, x[MAX_N];
4
5  int check(int k, int i) {
6
7  }
```

```

24 int main() {
25     n = 4;
26     n_queens(0);
27     return 0;
28 }
```

```

8 void n_queens(int k) {
9     int i;
10    if (k == n) {
11        for (i = 0; i < n; i++) {
12            printf("%d ", x[i]);
13        }
14        printf("\n");
15    } else {
16        for (i = 0; i < n; i++) {
17            if (check(k, i))
18            { x[k] = i;
19              n_queens(k + 1);
20            }
21        }
22    }
23 }
```

ฟังก์ชัน **check(k, i)** จะเช็คการชนกันของควีนตัวที่ **k** ในแถวที่ **i** กับ ควีนที่ถูกวางไว้ก่อนหน้านี้ โดยจะ **return 0** หากเกิดการชนกัน แต่ถ้าไม่ชนก็จะ **return 1**

```
8 int check(int k, int i) {  
9     for (int j = 0; j < k; j++) {  
10         if ( x[j] == i || abs(x[j] - i) == abs(j - k)) {  
11             return 0;  
12         }  
13     }  
14     return 1;  
15 }
```

สรุป

- อัลกอริทึม backtracking ถูกออกแบบมาเพื่อแก้ปัญหาของ brute force อย่างมีประสิทธิภาพ โดยกำหนดวิธีการเข้ารหัสคำตอบ และวิธีการสำรวจคำตอบทั้งหมดที่เป็นไปได้อย่างมีระบบ
- ในระหว่างการสำรวจเส้นทางของคำตอบจะมีการตัดเส้นทางของสถานะคำตอบที่ขัดแย้งกับเงื่อนไขบังคับของปัญหา เพื่อลดขนาดของ search space
- ข้อจำกัดของอัลกอริทึม backtracking
 - Exponential time complexity: หากคำตอบมีความลึกและกว้างมากๆ อาจส่งผลให้ประสิทธิภาพของอัลกอริทึมย่นรอยลดลงได้
 - Redundant or Duplicate Solutions: ความซ้ำซ้อนของคำตอบ